数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

September 30, 2026

Contents

1. 为什么 `testify/assert` 报 BrokenImport，以及怎么修 6
   1.1. 费曼直觉 6
   1.2. 硬核机制 6
      1.2.1. 四段解析链 6
      1.2.2. 三行报错信息的拆解 6
      1.2.3. 为什么 `go build ./...` 是绿的 7
      1.2.4. 最小复现 7
      1.2.5. 编辑器为什么“慢半拍” 7
   1.3. 陷阱与易错 8
   1.4. 关键点 8
   1.5. 变式检验 8
   1.6. 同类归类 8
   1.7. Go 语言机制补充 9
      1.7.1. 包：一个目录 = 一个包 9
      1.7.2. 导出：首字母大小写就是 public / private 9
      1.7.3. `internal/`：用目录位置表达的可见性围栏 10
      1.7.4. 为什么 `go build` 看不见测试文件 11
      1.7.5. module / package / import path：三个别混的术语 12
      1.7.6. 本机权威资料清单 12
2. go.mod 逐行讲解 14
   2.1. 费曼直觉 14
   2.2. 硬核机制 14
      2.2.1. 逐行拆解 14
      2.2.2. `go` 行的双重身份 15
      2.2.3. `// indirect` 是什么意思 16
      2.2.4. 模块图 ≠ 真正需要的模块 16
      2.2.5. 十条指令速查 17
   2.3. 陷阱与易错 18
   2.4. 关键点 18
   2.5. 变式检验 18
   2.6. 同类归类 18
   2.7. Go 语言机制补充 19
      2.7.1. MVS：版本是算出来的，不是问出来的 19
      2.7.2. 语义导入版本：`/v3` 不是可选装饰 19
      2.7.3. 工具链版本不用 semver 19
      2.7.4. go.sum 里到底记了什么 19
3. 为什么 `test/kv_test.go` 报 undefined: KV 20
   3.1. 费曼直觉 20
   3.2. 硬核机制 20
      3.2.1. 规范怎么规定的 20
      3.2.2. 改一次，错误往里走一层 20

3.2.3. 正确写法 21
3.3. 陷阱与易错 22
3.4. 关键点 23
3.5. 变式检验 23
3.6. 同类归类 23
3.7. Go 语言机制补充 23
3.7.1. 作用域是一组嵌套的块 23
3.7.2. 为什么 Go 不做“全局搜索” 24
3.7.3. 包名从哪来 24
4. 为什么不能 import “db/KV” 25
4.1. 费曼直觉 25
4.2. 硬核机制 25
4.2.1. 规范怎么说 25
4.2.2. 实测：四种写法各撞在哪 25
4.2.3. 正确写法 26
4.3. 陷阱与易错 26
4.4. 关键点 27
4.5. 变式检验 27
4.6. 同类归类 27
4.7. Go 语言机制补充 27
4.7.1. 三层概念：路径、包名、标识符 27
4.7.2. 为什么 Go 不学 Java 导入类 28
4.7.3. import 的四种形态 28
5. 为什么 kv.go 会报 missing function body 29
5.1. 费曼直觉 29
5.2. 硬核机制 29
5.2.1. 规范原文 29
5.2.2. 实测：三行签名，三个错 29
5.2.3. 为什么这个错一直没冒出来 29
5.2.4. 正确写法 30
5.3. 陷阱与易错 30
5.4. 关键点 30
5.5. 变式检验 31
5.6. 同类归类 31
5.7. Go 语言机制补充 31
5.7.1. 为什么 Go 不需要头文件 31
5.7.2. 编译是一张依赖链，不是一次全量扫描 31
6. 想分成 test 和 db 两部分，Go 社区怎么组织目录 33
6.1. 费曼直觉 33
6.2. 硬核机制 33
6.2.1. 先分清 module 与 package 33
6.2.2. 官方建议的几种结构 33

6.2.3. 两条容易记混的规则 34
6.2.4. 标准库自己的样子 34
6.2.5. 测试放哪儿：同目录的两种测试包 34
6.2.6. 你这个仓库的对照 35
6.3. 陷阱与易错 36
6.4. 关键点 36
6.5. 变式检验 36
6.6. 同类归类 36
6.7. Go 语言机制补充 36
6.7.1. `cmd/` 与 `internal/` 的性质完全不同 36
6.7.2. `testdata/`：工具链唯一“看不见”的目录 37
6.7.3. 从 GOPATH 到模块：目录观的转变 37
7. 用测试驱动开发，Go 的目录结构怎么摆 38
7.1. 费曼直觉 38
7.2. 硬核机制 38
7.2.1. 最小结构：一个包 38
7.2.2. 多包时的结构：包在哪，测试就在哪 38
7.2.3. TDD 循环在 Go 里长什么样（实测） 38
7.2.4. 两条命名规则 39
7.2.5. 常用命令速查 39
7.3. 陷阱与易错 40
7.4. 关键点 40
7.5. 变式检验 40
7.6. 同类归类 40
7.7. Go 语言机制补充 41
7.7.1. 为什么 Go 不需要单独的测试目录 41
7.7.2. 表驱动测试：社区的标志性写法 41
7.7.3. 没有内置的 watch 模式 41
8. 官方仓库里的 test/ 目录是怎么回事 43
8.1. 费曼直觉 43
8.2. 硬核机制 43
8.2.1. 两份“测试”的形态对照 43
8.2.2. 实测：那 392 个文件到底长什么样 43
8.2.3. 为什么它们必须独立存放 44
8.2.4. 对照：标准库包的测试在哪 44
8.3. 陷阱与易错 45
8.4. 关键点 45
8.5. 变式检验 45
8.6. 同类归类 45
8.7. Go 语言机制补充 45
8.7.1. 测试的三个层次 45
8.7.2. `testdata/`：被工具链无视的目录 46

9. go vet 通过 ≠ 能编译 47
9.1. 费曼直觉 47
9.2. 硬核机制 47
9.2.1. 实测现象 47
9.2.2. 为什么会有这个差异 47
9.2.3. 那 `go vet` 的价值在哪 48
9.2.4. 可靠的验证顺序 48
9.3. 陷阱与易错 49
9.4. 关键点 49
9.5. 变式检验 49
9.6. 同类归类 49
9.7. Go 语言机制补充 49
9.7.1. 为什么 Go 有两套类型检查器 49
Index of Listings 50

1. 为什么 `testify/assert` 报 BrokenImport，以及怎么修

1.1. 费曼直觉

GOPATH 时代的 Go 找包像**扫街**：沿着 `$GOPATH/src` 一条条路走过去，谁家门口挂着 `github.com/stretchr/testify` 的牌子，包就从谁家拿。模块模式把这个行为彻底关掉了，换成**登记制**——一个包要想被 import，必须先有人认领：

> 📌 **核心要点** `go.mod` 里的 `require` 就是认领登记表。表上没有这一行，Go 既不会去 `GOPATH` 里翻，也不会自作主张上网抓，它只会回一句“没有任何 required module 提供这个包”。

所以这句话不是“包坏了”，而是“**没人认领这个 import**”。理解了登记制，报错的字面意思就顺了：`no required module provides package` = 登记表里查无此人。

1.2. 硬核机制

1.2.1. 四段解析链

从一行 import 到真正拿到包，Go 要走完四段，任何一段断了都表现为“找不到包”，但文案不同：

1. **谁提供它**：import 路径 → 由 build list（受 `go.mod` 的 `require` 决定）选出的 module；
2. **哪个版本**：该 module 的版本在 `require` 里写死，`go get` 负责择优写回；
3. **完不完整**：`go.sum` 里的 `h1:` 哈希是模块内容的指纹，缺条目就是“下没下过都不知道”；
4. **拿不拿得到**：本地模块缓存（这个仓库里是 `/home/zz/go/pkg/mod`）没有时，才轮到 `GOPROXY` 出场。

这段链条解释了为什么同一个“找不到包”会有四种不同措辞——断点位置不同：

断点	命令行报错	编辑器（gopls）诊断
第一段：`go.mod` 无 require	`no required module provides package ...`	`could not import ...`；分类 `BrokenImport`
第二段：`go.mod` 里版本对不上	`missing go.sum entry for module ...`	同类 `BrokenImport`，detail 换成 go.sum 文案
第三段：校验和缺失或过期	`missing go.sum entry; to add it: go mod download ...`	编辑器与命令行口径一致，修好后消失
第四段：`GOPROXY=off` 且缓存无此模块	`module lookup disabled by GOPROXY=off`	同上，需先能拿到模块

1.2.2. 三行报错信息的拆解

原文是这四层叠起来的：

```
could not import github.com/stretchr/testify/assert (no required module provides package "github.com/stretchr/testify/assert") compiler (BrokenImport) [6, 2]
```

Listing 1: gopls 报错原文的四层结构

哪一层是谁说的，逐层拆开：

1. `could not import github.com/stretchr/testify/assert` ——**结论层**，gopls 的措辞；
2. `(no required module provides package ...)` ——**原因层**，直接把 go 命令的原话搬进来了；
3. `compiler` ——**来源层**，标注这条诊断出自 go/packages 的加载过程，而不是 gopls 自己的静态检查；
4. `(BrokenImport)` 与 `[6, 2]` ——**分类层与坐标**：`BrokenImport` 是“导入链断了”这一类，`[6, 2]` 是 `test/kv_test.go` 第 6 行第 2 列，也就是 `"github.com/stretchr/testify/assert"` 这个字符串字面量的起点（前导 tab 占掉第 1 列）。

1.2.3. 为什么 `go build ./...` 是绿的

这一点最误导人：`go build ./...` 压根**不编译** `_test.go` 文件。测试文件只在 `go test`、`go vet` 这类把测试纳入编译单元的命令里才会被拉进来。于是出现一个反直觉的组合：构建全绿，测试根本编不过。要复现，得让 `go vet ./test/` 或 `go test ./test/` 上场。

1.2.4. 最小复现

```
mkdir -p /tmp/probe && cd /tmp/probe
go mod init probe          # 只写 module + go 两行，没有 require
# 写一个 import "github.com/stretchr/testify/assert" 的 x_test.go
go vet ./...               # ⇒ no required module provides package ...; to add it:
                           #       go get github.com/stretchr/testify/assert
```

Listing 2: 复现缺失 require 时的原始文案

修复动作就是把第一段补上，一条命令同时写 `go.mod` 的 `require` 和 `go.sum` 的哈希：

```
cd /home/zz/project/db-go
go get github.com/stretchr/testify@latest   # 或 go mod tidy
go mod tidy                                 # 顺带清掉用不到的依赖
```

Listing 3: 补登记：go get / go mod tidy

1.2.5. 编辑器为什么“慢半拍”

gopls 是长驻进程，它在启动时把当时的 `go.mod` 解析成一份模块视图并缓存起来，之后靠文件监听去刷新。本仓库实测就撞上了这个错位：nvim 里的 gopls 进程起于 9 月 29 日 22:24，而 `go.mod` 补上 `require` 是 9 月 30 日 10:20——在它重新加载之前，磁盘上明明写好了，它眼里还是那张旧登记表。所以修完 `go.mod` 还有第二步：

```
:LspRestart          " nvim 里重启 LSP（最省事）
rm -rf ~/.cache/gopls  # 仍不生效时清缓存再重开 nvim
```

Listing 4: 让编辑器丢掉旧快照

⚠️ **陷阱预警** 报错里的“没有 required module”可能只是**编辑器视角**。下判断前先在终端跑一次 `go vet ./test/`：终端绿而编辑器红，问题在 gopls 的缓存，不在代码。

1.3. 陷阱与易错

1. **把 `go build ./...` 通过当成“代码没问题”**。它不碰 `_test.go`，测试能不能编译得用 `go vet ./test/` 或 `go test ./test/` 单独验；
2. **忘记提交 `go.sum`**。它记录的是模块内容哈希，少了它，别人克隆下来会直接卡在 `missing go.sum entry`。本仓库当前状态就是 `?? go.sum`（未跟踪）；
3. **只在编辑器里看着错误发呆**。gopls 的诊断位置 `[6, 2]` 指的不是 KV 类型，而是 import 那一行；把 import 修好之后才轮得到类型错误露头；
4. **修好 import 就以为测试能跑了**。这是两个独立问题，后者的坑在下一节。

⚠️ **陷阱预警** 这个仓库里 `test/` 目录是**黑盒测试**目录：文件声明 `package test`，而 `KV` 定义在 `internal/db` 包里，所以测试里必须写 `db.KV{}` 并 import `github.com/zz-cpp/db-go/internal/db`。裸写 `KV{}` 会得到 `undefined: KV`，那和 Broken-Import 无关。另外 `internal/db/kv.go` 目前全文只有一行 `package db` —— `KV` 类型本身还没实现，加了 `db.` 前缀也依然 undefined。

1.4. 关键点

📌 **核心要点** BrokenImport 问的是“谁认领”，不是“包坏没坏”：`go.mod` 的 `require` 是模块模式下的唯一登记表，`go get` / `go mod tidy` 补表，`:LspRestart` 让编辑器重读表。

1.5. 变式检验

1. **换环境**：如果 `go.mod` 里有 require、但项目里还有 `vendor/` 目录，行为会换成 vendor 模式——此时“找不到包”要查 `vendor/modules.txt` 和 `go mod vendor`，而不是 `go.sum`。断点还是四段链，只是第四段被换成了本地 vendor 树；
2. **换包名**：把 `testify/assert` 换成任意第三方包（比如 `github.com/google/uuid`），同样的 `no required module provides package` 会出现——说明触发条件是“import 了 go.mod 未登记的模块”这个**结构**，与具体是哪个包无关；
3. **换可见性**：如果 `internal/db` 不是 `internal` 而是普通包，别的模块也能 import 它；带 `internal` 时只有 `github.com/zz-cpp/db-go/...` 下的包能导入成功，这是**额外一层**、独立于模块登记的约束。

1.6. 同类归类

1. **同一类**：C/C++ 里 `#include "foo.h"` 找不到头的 `fatal error: foo.h: No such file or directory` ——都是“构建系统不知道去哪找依赖”，解法都落在“补搜索路径/补依赖声明”，而不是改代码；

2. **Python 类比**：Python 的 `ModuleNotFoundError` 与本节的 `no required module provides package` 是同一件事的两种说法：`requirements.txt` / `pyproject.toml` 就是 Python 的登记表；区别在于 pip 会真的去 PyPI 找，Go 在 `-mod=readonly` 下会先拒绝写 `go.mod`；
3. **往下延伸**：这正是本课程要亲手实现的东西——模块解析、版本选择、校验和校验，在 `go` 命令里是一个二进制内部的逻辑，在后面的存储引擎里会以“元数据文件 + 校验和”的形式重新出现（`internal/db/metadata.go` 的双槽设计与校验和就是同一个思想的小型复刻）。

1.7. Go 语言机制补充

上一节回答的是“怎么修”，这一节回答“为什么 Go 长这样”。对新手来说后者更值钱：三条命令只解决这一次，机制能迁移到下十个报错。下面每条结论都在本机实测过，出处给到能自己敲的路径。

1.7.1. 包：一个目录 = 一个包

Go 没有头文件，也没有 `#include`。它的组织单位是**包**：同一目录下所有 `.go` 文件用同一句 `package xxx` 声明自己属于哪个包，导入方写的是**导入路径**这个字符串，而不是文件名。

本仓库刚好展示了两种形态：

1. `internal/db/` 下每个文件都以 `package db` 开头——目录名 `internal/db` 与包名 `db` 不必相同（惯例相同，`internal` 这类纯组织目录是例外）；
2. `test/` 下每个文件都以 `package test` 开头——它是一个名叫 `test` 的**普通包**，`kv_test.go` 只是恰好带 `_test.go` 后缀的成员。

同包内标识符互相可见，跨包访问由下面两条规则管辖。

> 💡 **背景与类比** 出处：`go help packages`。想不起某条 go 命令的语义时，`go help <主题>` 是本机离线、且与当前版本严格对应的说明书；查标准库则用 `go doc <包>.<符号>`，例如 `go doc go/token.IsExported`。

1.7.2. 导出：首字母大小写就是 public / private

这是 Go 最省事也最容易绊人的设计。语言规范原文（本机 `/usr/lib/go/doc/go_spec.html`，Exported identifiers 一节）：

```
An identifier is exported if both:
  the first character of the identifier's name is a Unicode uppercase
  letter (Unicode character category Lu); and
  the identifier is declared in the package block
  or it is a field name or method name.
All other identifiers are not exported.
```

措辞是 **Unicode 大写字母（类别 Lu）**，不是“ASCII 的 A–Z”。这个细节对中文使用者有实际后果：汉字属于 Lo 类别，所以 `变量名` 这样的标识符**永远不会被导出**，不管它看起来多正式。标准库自带判定函数，可以直接验证：

```go
package main

import (
  "fmt"
  "go/token"
)

func main() {
  for _, s := range []string{"KV", "kv", "变量名", "Ωmega", "_x"} {
    fmt.Println(s, token.IsExported(s))
  }
}
```

Listing 5: 用标准库的判定函数实测导出规则；go/token.IsExported 的实现就是“首字符是否为 Unicode 大写字母”

实测结果：

标识符	首字符	是否导出	说明
`KV`	`K`	是	ASCII 大写字母，任何包都能访问
`kv`	`k`	否	小写，只有 `db` 包内部看得见
`变量名`	`变`	否	汉字属于 Lo 类别，永远导不出去
`Ωmega`	`Ω`	是	希腊字母的大写形式同样属于 Lu 类别
`_x`	`_`	否	下划线开头一律不导出，也是“故意不用”的占位习惯

于是上一节那句 `undefined: KV` 的谜底就清楚了：`KV` 定义在 `db` 包里、首字母大写，外部看得见，所以写法是 `db.KV`；换成小写的 `kv`，外部连它“存在”都看不到。

> ⚠️ **陷阱预警** 跨包访问未导出标识符时，编译器的原话是 `undefined: lib.hiddenFunc`，而不会说 “unexported” 或 “private”。原因是类型检查阶段的未导出标识符对导入方根本不可见，等价于“这个名字不存在”。看到 `undefined` 先别急着怀疑拼写——也可能是大小写写反了。

设计动机值得留意：C++/Java 用 `public` / `private` 关键字表达可见性，Go 把它换成命名约定。代价是“改个大小写就改变了 API 兼容性”这类隐蔽问题；收益是省掉一个关键字，并且强迫“要暴露出去的东西必须起个像样的名字”。

1.7.3. `internal/`：用目录位置表达的可见性围栏

Go 没有“模块私有”的语法，取而代之的是一条**纯目录位置**规则，定义直接写在 go 命令的实现里（本机 `/usr/lib/go/src/cmd/go/internal/load/pkg.go:1481`，`disallowInternal` 函数的注释）：

```
An import of a path containing the element "internal"
is disallowed if the importing code is outside the tree
rooted at the parent of the "internal" directory.
```

翻译成人话：导入路径里出现 `internal` 这一段时，只有以它**父目录**为根的树内部代码才能导入。本仓库 `github.com/zz-cpp/db-go/internal/db` 的父目录是 `github.com/zz-cpp/db-go`，于是规则等价于“只有 db-go 模块自己的包能导入它”。

实测两种情形（同一份代码，只换导入方）：

```
# ① 第三方模块视角：/tmp/internal-probe 有自己的 go.mod，用 replace 指向 db-go
$ cd /tmp/internal-probe && go build ./...
main.go:6:2: use of internal package github.com/zz-cpp/db-go/internal/db not allowed

# ② 同一模块内视角：db-go 自己的 test/ 目录
$ cd /home/zz/project/db-go && go vet ./test/
test/kv_test.go:10:8: undefined: KV
# 注意这里没有 internal 报错 —— 导入方在 db-go 树内，规则放行，报错停在下一步的类型检查
```

Listing 6: internal 规则的两次实测：跨模块被拦，同模块放行

> 📌 **核心要点** 这就是 `test/` 能 import `internal/db` 的原因：导入方 `github.com/zz-cpp/db-go/test` 位于 db-go 树内。换成任何第三方模块来导，构建阶段就直接失败。

设计动机：一个库往往希望“对外只暴露 API 包，实现细节不许被依赖”。Java 用 `module-info.java`、Rust 用 `pub(crate)`、Python 只有下划线约定（无强制力），Go 选了目录位置——零语法成本、零配置文件，把实现挪进 `internal/` 就收窄了可见性。轮到你自己写库时，这是最省事的封装手段。

1.7.4. 为什么 `go build` 看不见测试文件

上一节那个怪现象（构建全绿、测试编不过）的根源在**编译单元的划分**。`go help test` 的原文是：

```
'Go test' recompiles each package along with any files with names matching
the file pattern "*_test.go".
```

翻译过来：`go build` 的编译单元 = 目录里所有 `.go` 文件**减去** `*_test.go`；`go test` 才把它们拼进来，并额外链接出一个测试可执行文件。同一个 `go help test` 还顺手交代了两件新手常困惑的事。

第一件是**外部测试包**：

```
Test files that declare a package with the suffix "_test" will be compiled as a
separate package, and then linked and run with the main test binary.
```

也就是同目录下写 `package db_test` 的那些文件属于另一个包，只能访问 `db` 的导出标识符——同一个目录、不同的包，仅此一条规则，就强制你从使用者视角测试。

第二件是 `go test` 会顺带跑 vet：

```
As part of building a test binary, go test runs go vet on the package
and its test source files to identify significant problems.
```

这正是 `go vet ./test/` 能在不跑测试的情况下先暴露 BrokenImport 的原因。

本仓库的 `test/` 目录走的是另一条路：把测试放在独立目录、声明普通包名 `test`，再用 `import "github.com/zz-cpp/db-go/internal/db"` 拿到实现。效果同样是黑盒测试，但它不是 Go 的特殊机制，只是一次普通的跨包引用——所以它同时受 `internal` 规则和导出规则约束，两条都在上面讲过了。

1.7.5. module / package / import path：三个别混的术语

报错文案里 package 和 module 同时出现，新手最容易在这里打结。三者关系是：

概念	是什么	本仓库的实例
module（模块）	带 `go.mod` 的目录树，整体拥有一个版本号，是依赖管理的最小单位	`github.com/zz-cpp/db-go`，版本由 git tag 决定
package（包）	一个目录里的 `.go` 文件集合，是编译的最小单位	`db`（在 `internal/db/` 中）、`test`（在 `test/` 中）
import path（导入路径）	module path 拼上包目录的相对路径，代码里 `import` 写的就是它	`github.com/zz-cpp/db-go/internal/db`

`go help modules` 给出的定义是：

```
A module is a collection of packages that are released, versioned, and
distributed together.
```

那模块机制到底解决了什么痛点？GOPATH 时代所有代码只能躺在 `$GOPATH/src` 下，一个路径只能存在**一个版本**：想同时用同一个库的 v1 和 v2，做不到。模块让“每个项目自己声明依赖的版本”成为可能，`go.mod` 里的 `require` 一行由此而来——这也正是上一节那句 `no required module provides package` 的字面含义：没有任何**已登记**的 module 提供这个 package。

1.7.6. 本机权威资料清单

下面这些全都不需要联网，而且与你的 go1.27.1 严格对应：

想查什么	命令	源文件
语言规范全文	（只能直接读文件）	`/usr/lib/go/doc/go_spec.html`
语言内存模型	（只能直接读文件）	`/usr/lib/go/doc/go_mem.html`
任意标准库符号	`go doc <符号>` / `go doc -all <包>` / `go doc -src <符号>`	`/usr/lib/go/src/<包>/`
go 命令语义	`go help` + 主题：`modules` / `test` / `packages` / `environment`	`/usr/lib/go/src/cmd/go/internal/`
`internal` 规则实现	（只能直接读文件）	`/usr/lib/go/src/cmd/go/internal/load/pkg.go:1481`
标准库全部源码	（只能直接读文件）	`/usr/lib/go/src/`

📌 **核心要点** 以后遇到任何 Go 问题，按这个顺序自查：① `go doc <符号>` 看官方文档与示例 → ② `go doc -src <符号>` 看标准库是怎么实现的 → ③ `go help <主题>` 看工具链语义 → ④ 翻 `/usr/lib/go/doc/go_spec.html` 确认语言层面的边界。四步都在本机，不依赖网络，也不会因为查到“别的版本”的答案而走偏。

2. go.mod 逐行讲解

2.1. 费曼直觉

`go.mod` 是模块的**身份证**加**依赖登记表**：第一行回答“我是谁、别人怎么导入我”，其余各行回答“我需要谁、从哪个版本起”。

新手最容易带着 npm 的直觉来读它，以为 `require` 写的是“锁定的精确版本”。不是——`go.mod` 写的是**最小可接受版本**，最终用哪个版本由 `go` 命令在构建时用一套叫 MVS 的算法算出来。这是 Go 模块系统最核心的设计选择，最后一节展开。

各语言角色对照（注意 Go 没有单独的 lock 文件）：

语言	声明依赖的文件	版本怎么固定
Node	`package.json`	`package-lock.json` 锁死整棵依赖树
Python	`requirements.txt` / `pyproject.toml`	常靠 pip freeze 另存一份精确版本清单
Rust	`Cargo.toml`	`Cargo.lock` 锁死整棵依赖树
Go	`go.mod`	不锁：`require` 只写下限，具体版本由 MVS 推导；`go.sum` 只管**内容校验**，不管版本

2.2. 硬核机制

2.2.1. 逐行拆解

本仓库的 `go.mod` 只有四行有效内容（空行只是分组，无语法含义）：

```
module github.com/zz-cpp/db-go

go 1.27.1

require github.com/stretchr/testify v1.12.1

require go.yaml.in/yaml/v3 v3.0.5 // indirect
```

Listing 7: 本仓库 go.mod 的全部内容

`go.mod` 是纯文本，但有严格语法。`go mod edit -json` 能把解析结果原样倒出来，对照着看最清楚：

```
{
  "Module": {
    "Path": "github.com/zz-cpp/db-go"
  },
  "Go": "1.27.1",
  "Require": [
    {
      "Path": "github.com/stretchr/testify",
      "Version": "v1.12.1"
    },
    {
      "Path": "go.yaml.in/yaml/v3",
      "Version": "v3.0.5",
      "Indirect": true
    }
  ]
}
```

Listing 8: go mod edit -json：把 go.mod 摊成结构化字段，Indirect: true 就是那行注释的真实身份

四行逐一拆：

1. `module` **行——我是谁**。模块路径有三个用途：① 本模块内所有包的**导入路径前缀**（`internal/db` 目录里的包，完整导入路径就是前缀拼上 `/internal/db`）；② 依赖方引用本模块时的标识；③ 发布到模块代理时的地址。出处：`go help mod init`（`the module path for the new module`）。使用层面：**一旦发布就不要改**——模块路径是身份，改了等于换了个模块，下游所有 import 全部失效；
2. `go` **行——我要求哪个版本的语言**。它有两个身份，见下一小节；
3. **第一条** `require` **——直接依赖**：我的代码 `import` 了它提供的包；
4. **第二条** `require` **——间接依赖**：我的代码没有直接 import 它，是 `testify` 要用，见后面 `// indirect` 那一小节。

2.2.2. `go` 行的双重身份

`go 1.27.1` 既是**语言版本**声明（决定哪些语言特性可用、编译器按哪一版语义检查），也是**最低工具链要求**。后者是硬约束，实测如下：

```
$ go mod edit -go=1.99        # 把 go 行改成 1.99（本机只有 go1.27.1）
$ GOTOOLCHAIN=local go build ./...
go: go.mod requires go ≥ 1.99 (running go 1.27.1; GOTOOLCHAIN=local)
```

Listing 9: go 行是硬性下限：工具链版本不够，直接拒绝构建

（`GOTOOLCHAIN=local` 的作用是让 go 命令别去联网下载 1.99 的工具链，这样能看清“版本不足”这个错误本身；默认值 `auto` 会尝试自动获取缺失的工具链。）

写法上可以只写主次版本（`go 1.27`），也可以写到补丁级（`go 1.27.1`）——`go help mod edit` 的原话是 `It takes a version like "1.26" or "1.26.2"`。改它的推荐方式也不是手改文件，而是 `go get go@1.27.1`，因为同一段帮助里写了 `Users should prefer 'go get go@version'`。

💡 **背景与类比** 为什么语言版本要写进依赖文件？因为“旧代码在新工具链上继续能编”这句承诺的反面同样成立：语言语义会随版本变化（循环变量捕获规则、某些包的实现细节）。把版本写进 `go.mod`，等于让这份代码**声明自己按哪一版的规则解释**，工具链据此选择行为——老项目不会因为机器上装了新 Go 而突然改变语义。

2.2.3. `// indirect` 是什么意思

先看官方定义。`go help mod tidy` 在解释 tidy 如何决定保留哪些 require 时，给了一句关键的话：

```
(Go versions 1.17 and higher retain more requirements in order to
support lazy module loading.)
```

`lazy module loading`（惰性模块加载）就是动机：为了让构建时不必把整棵依赖树的 `go.mod` 全都下载解析一遍，Go 1.17 起把**更多** require 直接写进主模块的 `go.mod`，用空间换时间。被这样记录下来、但并非本模块代码直接 import 的模块，就标注 `// indirect`。

`go mod why -m` 能现场打印“为什么需要它”的完整链条：

```
$ go mod why -m go.yaml.in/yaml/v3
# go.yaml.in/yaml/v3
github.com/zz-cpp/db-go/test
github.com/zz-cpp/db-go/test.test
github.com/stretchr/testify/assert
github.com/stretchr/testify/assert/yaml
go.yaml.in/yaml/v3
```

Listing 10: 间接依赖的来龙去脉，从你的测试包一路读到 YAML 库

读法是从上往下：`test` 包被编成测试二进制 `test.test`，它 import 了 `testify/assert`；而 `assert` 里有个子包 `assert/yaml` 用到了 YAML 库。于是这个模块既不是你的直接依赖，又不能不在场。两种写法的区别归纳成表：

写法	含义
`require X v1.2.3`	本模块的代码直接 import 了 X 提供的某个包
`require X v1.2.3 // indirect`	本模块没有直接 import X，它是某个依赖的依赖；写进来是为了让惰性模块加载拿得到完整版本信息

⚠️ **陷阱预警** `// indirect` 只是**注释**，删掉它当次构建照样能过——但 `go mod tidy` 会把注释加回来，因为 go 命令会重新判断“这个模块是否被直接 import”。所以别手改这个标记，交给 `go mod tidy`。

2.2.4. 模块图 ≠ 真正需要的模块

`go mod graph` 打印的是**模块要求图**（谁 require 了谁），它甚至把 `go` 行也当成一个伪模块：

```
$ go mod graph
github.com/zz-cpp/db-go github.com/stretchr/testify@v1.12.1
github.com/zz-cpp/db-go go@1.27.1
github.com/zz-cpp/db-go go.yaml.in/yaml/v3@v3.0.5
github.com/stretchr/testify@v1.12.1 github.com/stretchr/objx@v0.5.3
github.com/stretchr/testify@v1.12.1 go.yaml.in/yaml/v3@v3.0.5
go@1.27.1 toolchain@go1.27.1
```

Listing 11: go mod graph 的真实输出：这是“谁声明了谁”，不是构建依赖清单

但图中 `objx` 那条线，本模块其实根本不需要：

```
$ go mod why -m github.com/stretchr/objx
# github.com/stretchr/objx
(main module does not need module github.com/stretchr/objx)
```

Listing 12: 图里有、实际不需要——这是两回事

`go.sum` 也印证了这一点，里面只有两个模块：

```
$ cut -d' ' -f1 go.sum | sort -u
github.com/stretchr/testify
go.yaml.in/yaml/v3
```

Listing 13: go.sum 记录的才是“会被读到的模块”

> 📌 **核心要点** 分清三个层次：**模块图**（谁声明了谁，看 `go mod graph`）⊃ **需要的模块**（构建与测试真正会读到的，由 `go mod why` 判定、被 `go.sum` 记录）⊃ **实际下载的源码**（躺在 `GOMODCACHE`，本机是 `/home/zz/go/pkg/mod/`，目录名格式为 `<模块路径>@<版本>`）。`objx` 只在第一层里。

2.2.5. 十条指令速查

`go.mod` 认得的全部指令（以 `go mod edit -json` 暴露的字段类型为准，即 Go 1.27 的集合）：

指令	一句话语义	本仓库
`module`	模块路径，即模块的身份	用
`go`	语言版本 + 最低工具链要求	用
`require`	登记一条依赖要求	用
`toolchain`	指定优先使用的工具链，默认由 go 行推导	没用
`replace`	把某个模块换成别的路径或本地目录，调试 fork 时常用	没用
`exclude`	排除某个具体版本，让它不参与版本选择	没用
`retract`	模块**作者**撤回自己已发布的版本，提醒使用者避开	没用
`godebug`	为兼容性开关指定 `GODEBUG` 默认值	没用
`tool`	声明本项目用到的开发工具依赖，如代码生成器	没用
`ignore`	告诉 go 命令忽略某些目录	没用

出处：`go help mod edit` 逐条解释了这些指令对应的编辑标志；`tool` 与 `ignore` 是较新加入的。

2.3. 陷阱与易错

1. **把 `go.mod` 当锁定文件**。“我明明写了 v1.12.1，为什么实际编进来的是更高版本”——因为 `require` 是下限，实际版本由 MVS 从整张要求图算出来；
2. **手改 `go.mod`**。推荐路径是 `go get`（增删改依赖）、`go mod tidy`（补齐与清理）、`go mod edit`（脚本化修改，官方定位就是 `for use primarily by tools or scripts`）。手改最容易漏掉 `go.sum` 的同步；
3. **删掉 `// indirect` 行想瘦身**。删掉会让构建找不到模块（回到上一节那类 BrokenImport 错误），因为惰性模块加载正是靠主模块的 `go.mod` 提供这些版本信息；
4. **以为 `go.sum` 是版本锁**。它存的是**内容哈希**（防篡改），不参与“选哪个版本”的决策；
5. **改 `module` 行做重构**。模块路径是身份，改了就换身份，全部导入路径随之失效；
6. **把 `/v3` 当可选的目录装饰**。它由语义导入版本规则强制，见下一节。

> ⚠️ **陷阱预警** 改完依赖记得同时检查三样东西是否一致：`go.mod`（要求）、`go.sum`（校验和）、`GOMODCACHE`（实际源码）。`go mod tidy` 会一次对齐前两者；只手动改了 `go.mod` 而 `go.sum` 没同步时，构建会停在 `missing go.sum entry`。

2.4. 关键点

> 📌 **核心要点** `go.mod` 只做两件事：声明身份（`module`）与声明**下限**（`go`、`require`）。版本的真实选择、源码的真实下载、内容的一致性校验，分别由 MVS、模块缓存与 `go.sum` 负责——分工明确，所以它能短到四行。

2.5. 变式检验

1. **换 vendor 模式**：仓库里出现 `vendor/` 目录后，go 命令会默认从 `vendor/` 读依赖、忽略 `GOMODCACHE` 里的副本，此时构建的事实来源是 `vendor/modules.txt`，`go.mod` 退居“声明”角色；
2. **换成本地替换**：`replace github.com/stretchr/testify => ../testify-fork` 后，`require` 里那条版本号只用于要求图，实际代码来自本地目录——这是调试第三方库的标准姿势；
3. **依赖要求冲突时**：若 A 要 v1.2.0、B 要 v1.5.0，MVS 选中 v1.5.0（同时满足两边的最小版本），既不报错也不回退——这点和“总是装最新版”的直觉相反，值得单独记住。

2.6. 同类归类

1. 与 `package.json`、`Cargo.toml`、`requirements.txt` 同类：都是“声明依赖的机器可读文件”，都在回答同一个问题——怎么让另一台机器复现出同样的构建；
2. 与 `go.sum` 的分工是 Go 特有的：声明与校验分离，前者管**要哪个版本**，后者管**拿到的内容是否与第一次一致**；
3. 往下延伸：本课程后面的 `metadata.go`（双槽 + 校验和）就是这套思路在存储层的复刻——记录“我期望的状态”与“我读到的状态”，并在两者不符时给出明确错误。

2.7. Go 语言机制补充

2.7.1. MVS：版本是算出来的，不是问出来的

MVS = Minimal Version Selection（最小版本选择），它的实现就在本机的 go 命令里（`/usr/lib/go/src/cmd/go/internal/mvs/mvs.go`），包注释只有两行：

```
// Package mvs implements Minimal Version Selection.
// See https://research.swtch.com/vgo-mvs.
```

规则一句话：**每个模块取所有要求里最高的那个下界**。因为每条 `require` 都是“至少这个版本”，所以满足全部要求的最小版本，就是这条链上最大的下界。收益是选择结果**可预测、可复现**（不需要 SAT 求解，不会今天装一套明天换一套）；代价是“想用最新版必须显式 `go get`”。

2.7.2. 语义导入版本：`/v3` 不是可选装饰

`go.yaml.in/yaml/v3` 里的 `/v3` 来自**语义导入版本**（Semantic Import Versioning）规则：主版本号 ≥ 2 时，**必须**在模块路径里带 `/vN` 后缀，包目录也要相应下移。设计动机很直接：v2 允许破坏性变更，若路径不变，下游就是“升级个依赖突然编不过”。把主版本写进导入路径之后，v1 与 v2 可以**同时**存在于同一份构建里，破坏性升级变成显式的代码改动（改 import 路径），不再隐形。

2.7.3. 工具链版本不用 semver

`go 1.27.1` 里的版本号有自己的规矩，`gover` 包的注释写得很直白：

```
// Package gover implements support for Go toolchain versions like 1.21.0 and 1.21rc1.
// (For historical reasons, Go does not use semver for its toolchains.)
```

Go 工具链版本历史上先于 semver 出现，于是保留了自己的一套（`1.27`、`1.27.1`、`1.27rc1`）。看到 `go 1.27.1` 别当普通 semver 去推兼容性保证。

2.7.4. go.sum 里到底记了什么

`go help module-auth` 的原文：

```
When the go command downloads a module zip file or go.mod file into the
module cache, it computes a cryptographic hash and compares it with a known
value to verify the file hasn't changed since it was first downloaded. Known
hashes are stored in a file in the module root directory named go.sum.
```

所以 `go.sum` 里每个模块各有两行是正常的：一行是**模块 zip 内容**的哈希（`h1:` 开头），一行是该模块 **`go.mod` 文件**的哈希（`/go.mod h1:` 开头）。`go.mod` 之所以单独记一次，是因为惰性模块加载下，构建可能只读 `go.mod` 而完全不下载整个 zip——只读它，就只需要它的哈希。

> 📌 **核心要点** 一句话收束：**`go.mod` 声明身份与下限，MVS 把下限算成具体版本，模块代理与本地缓存提供源码，`go.sum` 守住内容不被替换。** 四者分工明确，这正是 `go.mod` 能保持四行的原因。

3. 为什么 `test/kv_test.go` 报 undefined: KV

3.1. 费曼直觉

`undefined: X` 是编译器的“查无此名”。Go 有一条硬规矩：**每个名字都必须先声明、后使用**；查找路径也严格——从当前作用域一层层往外找，找到最外层的宇宙作用域（universe block，`int`、`true`、`len` 这类预声明名字就住在那儿）就停，**绝不会跨到别的包里乱翻**。

要使用别的包里的名字，必须“报上家门”，写成 `包名.名字`。你现在写的是 `KV{}`，等于在自家屋里喊了一声“KV”，而 `test` 这间屋子里没有这个人——于是 `undefined: KV`。

（顺带一提：报错里的路径是 `test/kv_test.go`，仓库里没有 `tests/` 目录，这个多出来的 s 大概只是手误。）

3.2. 硬核机制

3.2.1. 规范怎么规定的

语言规范（本机 `/usr/lib/go/doc/go_spec.html`）用两段话把规则讲全了。第一段是总则：

```
A declaration binds a non-blank identifier to a constant, type, type
parameter, variable, function, label, or package.
Every identifier in a program must be declared.
```

`Every identifier in a program must be declared` ——这一句就是 `undefined` 这个报错的全部由来。

第二段讲跨包引用，它给这种写法起了个专门的名字：**限定标识符**（qualified identifier）。

```
A qualified identifier is an identifier qualified with a package name prefix.
Both the package name and the identifier must not be blank.

QualifiedIdent = PackageName "." identifier .

A qualified identifier accesses an identifier in a different package, which
must be imported.
The identifier must be exported and
declared in the package block of that package.

math.Sin // denotes the Sin function in package math
```

拆开就是三个条件，缺一不可：① 那个包**必须被 import**；② 要访问的标识符**必须导出**（首字母大写，上一节讲过）；③ 它必须**真的声明在那个包的包块里**。

3.2.2. 改一次，错误往里走一层

在一份仓库副本上把三种改法依次做一遍（命令可复现），错误会像剥洋葱一样往里推：

```
$ go vet ./test/
vet: test/kv_test.go:10:8: undefined: KV

# 改法一：只加上 import，代码里仍写 KV{}
$ go vet ./test/
vet: test/kv_test.go:12:8: undefined: KV

# 改法二：把 KV{} 改成 db.KV{}，但 db 包里还没有 KV
$ go vet ./test/
vet: test/kv_test.go:12:11: undefined: db.KV

# 改法三：给 db 包补上 type KV struct{}
$ printf 'package db\n\ntype KV struct{}\n' > internal/db/kv.go
$ go vet ./test/
vet: test/kv_test.go:13:12: kv.Open undefined (type db.KV has no field or method Open)
```

Listing 14: 四次 go vet 的真实输出：每修一层，报错就往里推一层

四种状态归纳成表：

状态	报错	断在哪一层
现状：`kv := KV{}`	`undefined: KV`	当前包（`test`）里没有这个名字
加了 import，仍写 `KV{}`	还是 `undefined: KV`	import 不改变当前作用域——它只是把包“请进门”
写成 `db.KV{}`，包里没这个类型	`undefined: db.KV`	包名解析成功，成员解析失败
包里补上 `type KV struct{}`	`kv.Open undefined`	类型找到了，轮到方法——错误继续前移

注意改法二那次的报错位置 `12:11`：它指向的是 `KV` 而不是 `db`——因为 `db` 这个名字解析成功了（import 就在上面），失败的只是它在 `db` 包里的**成员**。这正说明限定标识符是**两段解析**。

3.2.3. 正确写法

要让 `undefined: KV` 彻底消失，得同时满足规范里那三个条件：

```go
package test

import (
  "testing"

  "github.com/stretchr/testify/assert"

  // ① 跨包访问前要先 import
  "github.com/zz-cpp/db-go/internal/db"
)

func TestKVBasic(t *testing.T) {
  kv := db.KV{}              // ② 跨包访问要带包名前缀
  err := kv.Open()           // ③ 前提是 db 包里真的声明了这些方法
  assert.Nil(t, err)
  defer kv.Close()
}
```

Listing 15: 三个条件在代码里的对应位置

但第 ③ 条是本仓库当前的瓶颈：`internal/db/kv.go` 全文只有一行 `package db`，`KV` 类型及其方法都还没被声明。README 里写得很清楚——**骨架，每个文件只有一行 `package` 声明，实现留给你写**。所以在 `internal/db/kv.go` 被填上之前，改 `test/kv_test.go` 只能把报错从“KV 未定义”推进到“db.KV 未定义”。

> 📌 **核心要点** `undefined: KV` 既不是语法错，也不是导入错，而是**名字解析**失败：编译器在你能看到的作用域里找不到这个名字。修的顺序永远是从外往内——先问“这个名字在哪个包”（补 import），再问“我写没写包名前缀”（写 `db.`），最后问“那个包里真的有它吗”（实现代码）。

3.3. 陷阱与易错

1. **一看到 `undefined` 就改拼写**。它有三种常见成因：拼错了、包前缀漏了、以及——那个名字所在的包还没实现。第三种在这个课程项目里是常态；
2. **以为 import 了就能直接用名字**。import 只建立“这个包可用”，不会把它的符号搬进当前作用域。好处是几乎不会有名字冲突：两个包都叫 `KV` 也不打架，写 `a.KV` 与 `b.KV` 即可；
3. **忘了 Go 的包作用域是“整目录级”的**。同一目录下所有文件共享一个包作用域：若 `test/` 里另一个文件声明了 `func helper()`，`kv_test.go` 可以直接调用，无需 import——包内可见的标准是“同一个包块”，与文件无关；
4. **import 了却用不上**。编译器会另报 `imported and not used`：Go 不允许导入未使用的包，这是语言规则而非 linter 偏好。

> ⚠️ **陷阱预警** 改这个文件时最容易忽略的是“第三个条件”：`internal/db/kv.go` 还是空骨架。别在 `test/kv_test.go` 里反复折腾——先用 `go doc ./internal/db` 看清那个包到底导出了什么，再决定怎么写测试。

💡 **背景与类比** `go doc` 不只能查标准库：`go doc ./internal/db`（或写全导入路径）会列出你自己这个包里所有**导出**的类型、函数与方法。写测试前先跑一次，就知道哪些名字真实存在——比反复编译试错快得多。

3.4. 关键点

📌 **核心要点** `undefined` = 名字解析失败。限定标识符 `包名.名字` 是两段解析：先解析包名（依赖 import），再解析成员（必须导出且确实声明过）。三段里断在哪一段，报错就停在哪一层——所以这个报错的位置和措辞，本身就是一张诊断地图。

3.5. 变式检验

1. **换到包内测试**：把测试文件挪进 `internal/db/` 并声明 `package db`，那么裸写 `KV{}` 就合法了——同一个包块内不需要前缀。这是黑盒测试与白盒测试在语法上的分水岭；
2. **换成未导出名**：在 `db` 包里定义 `type kvEntry struct{}`，外部写 `db.kvEntry{}` 同样得到 `undefined`——与本问题一模一样的报错，但成因是**导出规则**而非拼写。这两个坑长得极像，务必区分；
3. **换成大小写笔误**：把 `db.KV` 写成 `db.kv`，报错仍是 `undefined`。所以排查顺序应当是：先看大小写，再看包前缀，最后才怀疑“它到底实现了没有”。

3.6. 同类归类

1. 与 C 的 `error: 'foo' undeclared`、Python 的 `NameError: name 'foo' is not defined` 同类——都是名字解析失败，三种语言的差别只在查找规则与报错措辞；
2. 与上一节 BrokenImport 的关系是同一流水线上的两道工序：**导入解析**（go/packages 在加载阶段报）→ **类型检查**（编译器在分析阶段报）。前一道修好，才轮得到后一道显形；
3. 往下延伸：这正是编译器前端的经典三段——词法分析认出标识符、语法分析建出 AST、**类型检查做名字解析与类型绑定**。本课程后面要写的 SQL 解析器（`internal/db/sql_parser.go`）走的是同一条流水线，只是把被解析的语言从 Go 换成了 SQL。

3.7. Go 语言机制补充

3.7.1. 作用域是一组嵌套的块

规范把作用域定义成一串**块**（block）：宇宙块（universe，`int`、`len`、`true` 这类预声明名字）→ 包块（同一包的所有文件共享）→ 文件块（本文件的 import 名）→ 函数块 → 各种语句块（`for`、`if`、`switch`、`case` 各自开新块）。查找从最内层往外走，第一个命中的胜出——这就解释了“遮蔽”（shadow）行为：

```
var KV = 1          // 包级变量

func f() {
  KV := 2           // 内层块的同名声明，遮蔽了外面的 KV
  _ = KV            // 这里看到的是 2
}

```

Listing 16: 遮蔽：内层块的同名声明会挡住外层，这也是“名字解析从内往外”的直接后果

3.7.2. 为什么 Go 不做“全局搜索”

如果编译器允许“在所有包里搜这个名字”，立刻会有两个后果：名字冲突无法调和（两个包都有 `KV`，选谁?），构建速度也会崩（解析每个名字都要扫全部依赖）。Go 的选择是**显式优于隐式**：跨包必须写 `包名.`，而包名只能来自 import——于是查找范围被压到极小，这也是 Go 编译速度快的一个结构性原因。

3.7.3. 包名从哪来

`import "github.com/zz-cpp/db-go/internal/db"` 之后，代码里写 `db.` 而不是路径末段——因为**包名由源文件里的 `package` 子句决定，不由目录名决定**。默认情况下 import 引入的就是包自己声明的名字；想换叫法可以起别名：

```
import (
  db "github.com/zz-cpp/db-go/internal/db"    // 显式起别名，常用于避免重名
  _ "github.com/stretchr/testify"               // 只想要副作用（如驱动注册），不要名字
)

```

Listing 17: import 的两种特殊写法：别名与副作用导入

这也解释了本仓库的一个细节：目录叫 `internal/db`，包名却是 `db`——目录名只是路径的一部分，`db.` 这个前缀来自文件里的 `package db` 声明。

> 📌 **核心要点** 一句话收束：`undefined` **的含义是“在当前可见范围里找不到这个名字”。可见范围要么是当前包块（裸名），要么是“包名 + 该包的导出名”（限定名）。**修的时候按这个顺序想：包在不在（import）、前缀写没写（`db.`）、东西做没做（实现）。

4. 为什么不能 import “db/KV”

4.1. 费曼直觉

一句话：**Go 的 import 语句只能导入“包”，不能导入“类型”**。`KV` 是包里的一个类型，不是包本身——它没资格出现在 import 那对引号里面。

这个直觉通常来自 Java：`import com.example.db.KV;` 在 Java 里完全合法，因为 Java 的 import 导入的正是**类**。Go 不是这么设计的，这是两门语言分歧最大的地方之一——详细对比放在后面的机制补充里。

4.2. 硬核机制

4.2.1. 规范怎么说

语言规范里 import 的语法只有三行（本机 `/usr/lib/go/doc/go_spec.html`，Import declarations 一节）：

```
ImportDecl = "import" ( ImportSpec | "(" { ImportSpec ";" } ")" ) .
ImportSpec = [ "." | PackageName ] ImportPath .
ImportPath = string_lit .
```

盯着第二行：`ImportSpec` 只有两个位置——可选的 `"." | PackageName`（点或别名），以及 `ImportPath`（一个字符串字面量）。**语法里根本没有“类型名”的位置**。所以写在引号里的 `db/KV` 会被整个当作一个**导入路径**去解析，而不是“包 `db` 里的 `KV`”。

同一节还给了一句定义，措辞很关键：

```
The import names an identifier (PackageName) to be used for access and an ImportPath
that specifies the package to be imported.
```

import 声明建立的是“**一个用于访问的标识符（包名）+ 一个指向包的路径**”——从头到尾说的都是**包**。

4.2.2. 实测：四种写法各撞在哪

在一份仓库副本上把这几种写法都试了一遍（不动你的工作区）：

写法	`go vet ./test/` 报什么	Go 把它当成什么
`import "db/KV"`	`package db/KV is not in std (/usr/lib/go/src/db/KV)`	标准库包——路径首段没有点号，Go 先去标准库找
`import "db"`	`package db is not in std (/usr/lib/go/src/db)`	同上：砍掉后半段也救不了
`import "…/db-go/db"`	`no required module provides package …`	模块内的包，但路径拼错了（真实目录是 `internal/db`）

`import . "…/internal/db"` + 裸写 `KV{}`	`kv.Open undefined` ——说明 `KV` 已解析成功	点导入：符号直接进了本文件作用域

前两行的报错特别能说明问题：`db/KV` 的首段没有点号，Go 判定“这大概是标准库”，于是去 `/usr/lib/go/src/` 底下翻——那里当然没有叫 `db` 的包。第三行则揭开了导入路径的构造规则：**模块路径 + 包目录的相对路径**；本模块是 `github.com/zz-cpp/db-go`，包在 `internal/db` 目录里，所以完整路径只能是这两段拼接。（那行报错的后半句是 `to add it: go get …`，也就是 go 在建议你 `go get` 一个并不存在的路径。）

第四行是个例外发现：点导入确实能让裸写 `KV{}` 生效——它的代价见后文。

4.2.3. 正确写法

```
package test

import (
  "testing"

  "github.com/stretchr/testify/assert"

  "github.com/zz-cpp/db-go/internal/db"    // 导入的是包：模块路径 + 目录相对路径
)

func TestKVBasic(t *testing.T) {
  kv := db.KV{}                            // 用的时候才出现类型名：包名.类型名
  err := kv.Open()                         // 前提是 db 包里真的声明了 KV 与 Open
  assert.Nil(t, err)
  defer kv.Close()
}
```

Listing 18: 包从 import 进来，类型名出现在使用处——两个位置分工明确

📌 **核心要点** 把这三样东西分清楚，这类问题就再也不会犯：**导入路径**（字符串，如 `github.com/zz-cpp/db-go/internal/db`，用来找包）→ **包名**（如 `db`，由对方文件里的 `package` 子句决定，当作前缀用）→ **标识符**（如 `KV`，包里的符号）。`import "db/KV"` 的错误在于把“包”和“包里的类型”塞进了同一个字符串。

4.3. 陷阱与易错

1. **路径写成本地相对路径。** `import "db"`、`import "./db"`、`import "../db"` 都不成立：Go 的导入路径要么是标准库名（如 `fmt`），要么是“模块路径 + 目录”，从不使用文件系统的相对路径；
2. **以为目录名就是导入路径。** 本仓库目录是 `internal/db`，路径是 `github.com/zz-cpp/db-go/internal/db`——前半段的模块路径来自 `go.mod` 的 `module` 行，不是自动推导的；
3. **给 `internal` 包写“短路径”。** `internal` 只影响**谁能**导入（上一节讲过），不影响路径怎么写；

4. **一个包 import 两次**。同一个包在同一个文件里重复 import 会报错；但不同文件各自 import 同一个包是正常的。

> ⚠️ **陷阱预警** 路径写错时，报错措辞会随“路径形状”变：**首段无点号**（如 `db`、`db/KV`）→ 报 `is not in std`（Go 当它是标准库）；**带域名**（如 `github.com/x/y`）→ 报 `no required module provides package`（Go 当它是模块依赖）。两种报错都指向“路径不存在”，与“类型不存在”是两回事。

4.4. 关键点

> 📌 **核心要点** Go 的 import 引入的是**包**，不是符号：`import "路径"` 只做一件事——把那个包的**包名**登记进本文件的作用域。要访问包里的类型或函数，永远写成 `包名.名字`。这也是为什么 Go 的 import 列表天然很短：一个包导入一次，之后用多少个符号都不用再改 import。

4.5. 变式检验

1. **换包别名**：`import d "github.com/zz-cpp/db-go/internal/db"` 后写 `d.KV`——当包名与已有标识符冲突、或包名太长时用；
2. **换点导入**：`import . "github.com/zz-cpp/db-go/internal/db"` 后可以直接写 `KV{}`，实测确实可行。**但别这么写**：任何名字都看不出出处，与本地声明撞名时极难排查，而且 `goimports` 之类工具也没法帮你整理；
3. **换到包内测试**：把测试文件放进 `internal/db/` 并声明 `package db`，那么既不用 import 也不用前缀——这是三条路里最干净的一条，代价是失去了“只测导出 API”的约束。

4.6. 同类归类

1. 与 Java 的分歧最大：Java 的 `import com.example.db.KV;` 导入的是**类**，之后直接写 `KV`；Go 的 import 导入的是**包**，之后写 `db.KV`。写 Java 时记住“import 类”，写 Go 时记住“import 包”；
2. 与 Python 的关系更近但也不同：Python 的 `import db` 之后同样写 `db.KV`，看起来像；但 Python 的模块是**文件**，Go 的包是**目录**，而且 Python 还能 `from db import KV` 直接把符号搬进来——Go 没有等价的常规写法（只有点导入这个“非常规武器”）；
3. 往下延伸：本课程后面要实现 SQL 的词法与语法分析，那时也会遇到“名字怎么解析”的同类问题——SQL 里的表名、列名同样要在某个作用域里查找，作用域设计错了，后面全是补丁。

4.7. Go 语言机制补充

4.7.1. 三层概念：路径、包名、标识符

这是本节的核心，值得单独成表：

层	是什么	本仓库的例子

导入路径	字符串，唯一标识一个包，用于 import 和依赖管理	`github.com/zz-cpp/db-go/internal/db`
包名	代码里使用的前缀，由被导入包的 `package` 子句决定	`db`
标识符	包内声明的符号，首字母大写才导出	`KV` 、 `Open` 、 `Set` ……

三者可以互相“错位”而不出问题：目录叫 `internal/db` 、路径后缀是 `internal/db` 、包名却是 `db` ——因为包名只认文件里的 `package` 声明。这也是为什么有时候你会看到 `import "github.com/user/repo/v2/pkg"` 用起来却是 `pkg.XXX` 。

4.7.2. 为什么 Go 不学 Java 导入类

四个理由，按重要性排：

1. **显式优于隐式**：代码里 `db.KV` 一眼看出它来自哪个包；Java 里看到裸 `KV` ，得翻回 import 区才知道是谁家的；
2. **冲突天然可解**：两个包都有 `KV` ，Go 写 `a.KV` / `b.KV` 即可，不需要改名或全限定名；
3. **import 列表稳定**：多用一个符号不需要动 import——Java 每引入一个类就要加一行 import；
4. **编译速度**：`db.KV` 的解析只需查一个已知包的符号表，不需要在所有已导入的包里做全局名字搜索。

4.7.3. import 的四种形态

```
1     import "fmt"                                    // 常规：用包自己的名字 fmt
2     import f "fmt"                                  // 别名：之后写 f.Println
3     import . "fmt"                                  // 点导入：符号直接可见，
写 Println
4     import _ "github.com/lib/pq"                    // 空标识符：只想执行包的 init，
如注册数据库驱动
```

Listing 19: import 的四种写法；第三种合法但不推荐，第四种常见于驱动注册

第四种（下划线）值得一提：它叫**副作用导入**，唯一目的是触发那个包的 `init` 函数——数据库驱动就是经典场景（驱动在 `init` 里把自己注册进 `database/sql` ）。本课程后面如果要连真实数据库，会再见到它。

> 📌 **核心要点** 一句话收束：**Go 的 import 是“请一个包进门”，不是“搬一个类型进来”。**路径负责“找谁”，包名负责“怎么称呼”，标识符负责“要哪个东西”——三件事由三处语法各管一段，混在一起就必然报错。

5. 为什么 kv.go 会报 missing function body

5.1. 费曼直觉

在别的语言里“先写签名、后补实现”是家常便饭：C 用头文件声明、Java 写 `abstract`、TypeScript 写 `declare`。**Go 没有这套分层**——它没有头文件，函数声明就必须连身体一起给。

唯一的例外是“身体不在 Go 里”：函数由汇编之类的语言实现时，才可以只留签名。这就是规范里那个可选的 `[ FunctionBody ]` 的全部含义。

5.2. 硬核机制

5.2.1. 规范原文

```
FunctionDecl = "func" FunctionName [ TypeParameters ] Signature [ FunctionBody ] .
FunctionBody = Block .

A function declaration without type parameters may omit the body.
Such a declaration provides the signature for a function implemented outside Go,
such as an assembly routine.

func flushICache(begin, end uintptr)  // implemented externally
```

注意 `[ FunctionBody ]` 是方括号——语法上确实“可省”。但紧跟着的那句把条件限死了：**只有当函数由 Go 之外的东西实现时**才允许。随手省掉函数体不属于这个例外。

5.2.2. 实测：三行签名，三个错

`internal/db/kv.go` 里 `Get`、`Set`、`Del` 三行只写了签名（第 14–16 行），`go build ./...` 立刻报出：

```
$ go build ./...
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:14:6: missing function body
internal/db/kv.go:15:6: missing function body
internal/db/kv.go:16:6: missing function body
```

Listing 20: missing function body：编译器在解析阶段就拒绝，连类型检查都没走到

有意思的是它的位置标记：`14:6` 指向的不是方法名，而是接收者那对括号的起点——编译器是在读到“签名结束却没等到 `{` ”时放弃的。

5.2.3. 为什么这个错一直没冒出来

你可能注意到：在同一时刻，`go vet ./test/` 只报了 `undefined: KV`，对 `kv.go` 的语法错误一字未提。原因是**编译单元**：

1. `test` 包当时**还没有 import** `internal/db`，所以编译 `test` 包时根本不需要碰 `db` 包——`db` 包的错误自然“藏”着；
2. 一旦补上 import，`db` 包被拉进依赖链，错误立刻现形。我在副本上实测过：

```
# 补上 import "github.com/zz-cpp/db-go/internal/db" 与 db.KV{} 之后
$ go vet ./test/
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:14:6: missing function body
internal/db/kv.go:15:6: missing function body
internal/db/kv.go:16:6: missing function body
```

Listing 21: 同一个命令，加了 import 之后报的东西完全不同——依赖把错误带进了视野

这与前面几节其实是同一条规律：`go build` 不编译 `_test.go`、未登记的模块不会去下载、没有 import 的包不会被编译——**编译器只碰它需要的东西**。所以“编译通过”永远只是“我碰过的那部分没问题”，想看得更全就得让它多碰一点（`go build ./...`、`go vet ./...`）。

5.2.4. 正确写法

给方法补上函数体即可。以 `Get` 为例，用 `KV` 已有的那个 map 字段写最短实现：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok = kv.mem[string(key)]
  return val, ok, nil
}
```

Listing 22: 补上函数体后的 Get：从 map 取值，找不到时 ok 为 false 而不是报错

注意两处细节：map 的键必须是可比较类型，所以 `[]byte` 要转成 `string`（这次转换是**零拷贝**的语义特例，编译器不会真的复制一遍）；“找不到”用 `ok = false` 表达，而不是返回错误——这正是 `test/kv_test.go` 第 21–22 行断言的行为。

> ⚠️ **陷阱预警** **不要**靠“留白函数体”占位，也不要写 `func (kv *KV) Get(...) { return }` 来骗过编译——后者虽然合法，但当函数有具名返回值时会静默返回零值，把“还没实现”伪装成“返回空结果”，调试时会非常难找。

5.3. 陷阱与易错

1. **把方法签名当“接口实现声明”**。在 `interface` 里只写签名是完全合法的（接口声明的是契约）；在具体类型上只写签名则非法。同样是“没有函数体”，合法与否取决于写在哪儿；
2. **以为 `go vet ./test/` 过了就万事大吉**。它只检查你点名的包及其依赖，没被 import 的包一律不看。想全覆盖用 `go vet ./...` 或 `go build ./...`；
3. **被“缺失函数体”的报错行号误导**。它标在接收者括号处（`14:6`），而不是方法名，别顺着行号去找错字。

5.4. 关键点

> 📌 **核心要点** Go 没有头文件，**声明与定义必须合一**：函数要么带 `{}` 函数体，要么其实现存在于 Go 之外（汇编）。`missing function body` 是解析阶段就抛出的错误——它比类型错误更早，说明编译器连语法树都没能建完。

5.5. 变式检验

1. **换成接口**：`type Reader interface { Read(p []byte) (n int, err error) }` 只有签名没有体，完全合法——接口描述“能做什么”，此时无体才是正确形态；
2. **换成汇编实现**：把函数放到同包的 `.s` 文件里实现（标准库大量使用，如 `runtime.memmove`），此时 Go 文件里可以只留签名，编译器接受；
3. **换成语义等价的手写桩**：`func (kv *KV) Set(key, val []byte) (bool, error) { panic("not implemented") }`——占位时可以，但要在提交前换掉，避免把“忘了实现”带进构建。

5.6. 同类归类

1. 与 C/C++ 的对照最直接：C 把“声明”放进 `.h`、把“定义”放进 `.c`，所以“先声明后实现”天经地义；Go 取消了头文件，代价就是声明必须自带实现；
2. 与 Java 的对照：Java 的 `abstract` / `native` 方法也只有签名，但那要求类为 `abstract` 或用 JNI——同样有明确前提，不是随手可省；
3. 往下延伸：本课程后面要实现自己的接口抽象（比如存储引擎分层），会用到 Go 的 `interface`——那时“只有签名”就变成正规写法了，正好与本问题形成对照。

5.7. Go 语言机制补充

5.7.1. 为什么 Go 不需要头文件

头文件解决的是“编译单元之间互相看不到”的问题：C 的编译器一次只处理一个 `.c`，所以需要 `.h` 告诉它别处有什么。Go 的编译器直接读源码、按包组织，导出信息（哪些名字首字母大写）从源码本身就能提取，于是头文件这一层被整个删掉了。规范里那句“省略函数体 = 实现在 Go 之外”，正是这个设计的补丁：唯一的例外留给真正跨语言的场景。

5.7.2. 编译是一张依赖链，不是一次全量扫描

从以上实验能提炼出 `go` 命令的工作模型：你给它一个起点（包），它顺着 import 图把需要的包拉进来，逐个编译——**没被依赖到的包完全不参与**。这条规律能解释三个此前遇到过的现象：

现象	原因
`go build ./...` 通过，`go vet ./test/` 却报错	`_test.go` 不在 `go build` 的编译单元里
`go vet ./test/` 只报 `undefined: KV`，不报 `kv.go` 的语法错	`test` 包当时没有 import `db` 包，`db` 包不在依赖链上
模块没写进 `go.mod` 就报 `no required module provides package`	依赖图里没有它，连下载这一步都不会发生

> 📌 **核心要点** 一句话收束：`missing function body` **是“你只写了承诺，没写兑现”**。Go 的声明与定义不分离——要么给 `{}`，要么承认这段代码在 Go 之外。顺带记住那条

> 更通用的规律：编译只走依赖链，所以想让编译器替你查出更多问题，就用 `./...` 把范围指全。

6. 想分成 test 和 db 两部分，Go 社区怎么组织目录

6.1. 费曼直觉

先纠正一个术语：你想分的“两个模块”，在 Go 里其实叫“两个**包**”。这不是抠字眼，因为它决定了你该动的是目录，而不是 `go.mod`。

Go 的组织方式很朴素：**目录即包**。一个目录里的 `.go` 文件同属一个包，导入路径就是这个目录的位置。所以“怎么分代码”这个问题，在 Go 里等价于“怎么摆目录”。

6.2. 硬核机制

6.2.1. 先分清 module 与 package

概念	是什么	本仓库
module 模块	带 `go.mod` 的目录树，有版本号，是**发布与依赖管理**的单位	`github.com/zz-cpp/db-go`（整个仓库一个）
package 包	一个目录里的 `.go` 文件集合，是**编译与导入**的单位	`db`、`test`、`main`（仓库里有好几个）

一个 module 里通常有多个 package——你现在就有三个（`cmd/dbshell` 的 main、`internal/db` 的 db、`test` 的 test）。**要“分成两个模块”意味着拆出第二个 `go.mod`**，那是一种很重的操作，通常只在“要独立发布、独立版本号”时才做（多模块需要 `go.work` 把它们串起来）。你想要的显然是分成两个**包**，那就只需摆目录。

6.2.2. 官方建议的几种结构

权威出处是官方文档 `go.dev/doc/modules/layout`（Organizing a Go module）。它的立场很明确：**Go 没有强制的项目布局**，只有随项目规模递进的建议。把其中与“多包”相关的几条摘出来：

项目形态	目录结构	要点
单个包	代码全放根目录	`go.mod` + `modname.go` + `modname_test.go`
单个命令	根目录多个文件同属 `package main`	安装路径最短：`go install …/modname@latest`
包 + 支撑包	支撑包放 `internal/`	外部模块导不进来，API 不必对外承诺
多个包	每个包一个子目录，可分层	`auth/`、`auth/token/`、`hash/` 平铺或嵌套
多个命令	每个命令一个目录，共享代码进 `internal/`	`prog1/main.go`、`prog2/main.go`
包与命令混合	命令集中到 `cmd/`，可导出包留在顶层	本仓库用的就是这种

6.2.3. 两条容易记混的规则

1. `internal/` **是编译器强制的**：路径里含 `internal` 的包，只有以其父目录为根的树内代码能导入。这是 Go 里唯一被工具链特殊对待的目录名（前面有一节专门讲过它的实现）；
2. `cmd/` **只是惯例，不是规则**：每个子目录放一个 `package main`，子目录名通常等于可执行文件名。把它换成别的名字也能编译，只是没人这么干。

> ⚠ **陷阱预警** 搜“Go 项目结构”时，你大概率会搜到 `github.com/golang-standards/project-layout`。**它不是官方标准**——仓库自己的 README 就写明了只收录社区常见模式；Russ Cox 也曾指出用 `golang-standards` 这个组织名有冒称官方之嫌。它推荐的 `pkg/` 目录尤其有争议，官方文档里根本没有 `pkg/`。找权威说法请认准 `go.dev/doc/modules/layout`。

6.2.4. 标准库自己的样子

GOROOT 里的标准库是 Go 团队自己的代码，它的组织方式就是最有说服力的样本（本机 `/usr/lib/go/src/`）：

```
/usr/lib/go/src/
├── cmd/          ← 可执行命令：go、compile、vet...
├── internal/     ← 不对外暴露的实现
├── net/http/     ← 一个公共库包
├── runtime/
└── ...            ← 其余标准库包，每个占一个目录
```

Listing 23: 标准库顶层：cmd/ 放命令，其余目录各自是一个包

再往里看一层，`net/http` 目录里的测试文件是这样组织的——注意两种包名同时存在：

```
$ grep -l '^package http$'      /usr/lib/go/src/net/http/*_test.go | wc -l
19
$ grep -l '^package http_test$' /usr/lib/go/src/net/http/*_test.go | wc -l
19
```

Listing 24: net/http 目录里：19 个内部测试文件 + 19 个外部测试包文件，同在一个目录

也就是说：**官方自己既不把测试挪到单独的 test/ 目录，也不把它们塞进一个包**——两种测试包就在被测代码旁边。

6.2.5. 测试放哪儿：同目录的两种测试包

这才是你真正想问的那件事。Go 的惯例是：测试文件与被测代码**同目录**，文件名加 `_test.go` 后缀，包名有两种选择：

声明	含义与能力边界
`package db`	内部测试：和被测代码同属一个包，能访问未导出标识符，适合测内部细节
`package db_test`	外部测试包：独立包，只能访问导出 API，强制你从使用者视角测试

`go help test` 的原文给它撑腰：`Test files that declare a package with the suffix "_test" will be compiled as a`
我在副本上实测过这套组织方式（实现放 `internal/db/kv.go`，黑盒测试放同目录的 `internal/db/kv_test.go` 声明 `package db_test`）：

```
$ go test ./internal/db/
ok    github.com/zz-cpp/db-go/internal/db 0.002s
```

Listing 25: 外部测试包与实现同目录，go test 正常工作

边界也很清晰——一旦外部测试包想碰私有字段，报错会明确告诉你原因：

```
$ go test ./internal/db/
internal/db/kv_test.go:11:9: kv.mem undefined (cannot refer to unexported field mem)
FAIL  github.com/zz-cpp/db-go/internal/db [build failed]
```

Listing 26: cannot refer to unexported field：外部测试包只能看导出 API

> 📌 **核心要点** 想同时得到“测试与实现分开”和“符合 Go 惯例”两个结果，正确做法是**外部测试包**：测试文件留在 `internal/db/` 目录里，但声明 `package db_test`。它天然是“另一个包”（拿不到私有成员），却不需要额外的目录。这正是标准库在用的方式。

那么 `test/` 目录在社区里什么时候用？确实有一种正当用法：**端到端测试与集成测试**。像 Kubernetes 那样的大型项目会把跨组件的 e2e 测试集中放 `test/e2e/`，因为它们测的不是某一个包，而是整个二进制。你现在的 `test/kv_test.go` 测的是单一类型的行为，属于单元测试，放在 `test/` 目录偏离了惯例。

6.2.6. 你这个仓库的对照

目录	现状	是否合惯例
`cmd/dbshell/`	命令行入口 `main` 包	合——标准的 `cmd/` 用法
`internal/db/`	实现包 `db`	合—— `internal` 阻止外部导入，且包名与目录名一致
`test/`	测试文件 `package test`	不合——单元测试按惯例应与被测代码同目录

如果想对齐惯例，改动很小：

1. 把 `test/kv_test.go` 移到 `internal/db/kv_test.go`，包名从 `package test` 改成 `package db_test`；
2. 然后 `kv := KV{}` 恢复成 `kv := db.KV{}` 并 import 那个包（对了，那个文件现在还没 import db 包）；
3. `test/` 目录删掉，或留作日后的 e2e 测试。

代价也要说清楚：课程讲义若按 `test/` 目录讲，改动会让你与讲义对不上号。所以这不是“必须改”，而是“知道惯例长什么样，再决定跟不跟”。

6.3. 陷阱与易错

1. **以为“目录名 = 包名”是强制的**。目录叫 `internal/db`、包名是 `db` 完全正常；强制的只有“同一个目录里只能有一个普通包名”；
2. **在同一目录里放两个包**。除了 `foo` 与 `foo_test` 这一对特例，其余都会报 `found packages a and b in …`；
3. **为了“分层”造 `util/`、`common/`、`helper/` 包**。这类包会退化成“代码垃圾场”——官方与社区都建议按语义命名（`text.Reverse` 比 `util.Reverse` 好）；
4. **过早套用完整模板**。官方立场是布局随项目增长而演进：一个 200 行的小工具不需要 `cmd/` 和 `internal/`；
5. **把 `pkg/` 当标准**。它来自社区那个非官方仓库，官方文档里没有这个目录。

6.4. 关键点

> 📌 **核心要点** Go 的代码组织只有一条物理规则：**目录即包**。在此之上，`internal/` 是编译器强制的私有区，`cmd/` 是社区约定俗成的命令区，测试则按惯例与被测代码同目录——想“分成两个包”，用 `package db` 与 `package db_test` 这一对就够了，不必新造目录。

6.5. 变式检验

1. **换成为第三方提供库**：把包从 `internal/db` 挪到 `db/`（去掉 internal），外部模块就能 import 它——这时 `internal` 的取舍就是“要不要对外承诺 API”的取舍；
2. **换成多个可执行程序**：再加 `cmd/dbctl/main.go` 即可，两个命令共享 `internal/db`——这正是官方第 6 种结构的形态；
3. **换成真需要两个 module**：若 `test` 那部分要独立发布、独立版本号，就得拆出第二个 `go.mod`，并用 `go work init` 建立工作区把它们串起来。复杂度陡增，非必要不做。

6.6. 同类归类

1. 与 Java 的分野：Java 用包名（`com.example.db`）映射到目录，但物理位置由 `-sourcepath` / 构建工具决定；Go 把“目录即包”写死进工具链，导入路径就是目录位置；
2. 与 Python 相近处：都是一个目录一个模块/包、都有 `__init__.py` 与 `package` 声明之别；但 Python 的导入路径基于文件系统（且可在运行时改），Go 基于模块路径（编译期固定）；
3. 往下延伸：本课程后面要做的存储引擎会有多层（KV 层、SSTable 层、关系表层），届时你会真正面对“怎么切包”的问题——那时请记住两条：**按职责切，不按类型切**（不要建 `types/`）；**先平铺，痛了再拆**。

6.7. Go 语言机制补充

6.7.1. `cmd/` 与 `internal/` 的性质完全不同

新手常把这两个目录当作同一类“官方约定”，其实差别很大：

目录	性质	违反后果

`internal/`	编译器强制：路径含 `internal` 时，只有父目录树内的代码可导入	构建失败：`use of internal package … not allowed`
`cmd/`	纯社区惯例：放 `package main` 的目录	什么也不会发生，只是别人看不懂

6.7.2. `testdata/`：工具链唯一“看不见”的目录

还有一个特殊目录值得记住：名叫 `testdata` 的目录会被 go 工具链**完全忽略**（`go help test` 原文：`The go tool will ignore a directory named "testdata", making it available to hold ancil data needed by the tests`）。

于是它成了存放测试夹具（fixture）的标准位置——本课程后面做 SSTable 时，需要放二进制测试数据文件，那里就是它们的家。

6.7.3. 从 GOPATH 到模块：目录观的转变

`go help gopath` 里还留着上一代的痕迹：GOPATH 模式下，代码必须住进 `$GOPATH/src/github.com/user/repo`，导入路径由它在磁盘上的位置推导出来。模块模式取消了这层嵌套——**导入路径由 `go.mod` 的 `module` 行加目录相对路径决定，与仓库放在磁盘哪里无关**。这也是为什么现在的项目结构可以更自由，但“目录即包”这条没变。

> 📌 **核心要点** 一句话收束：**分代码就是分目录，分目录就是分包。** 对外可导入的包放顶层，不想被依赖的丢 `internal/`，命令丢 `cmd/`，测试与被测代码同目录（要黑盒就声明 `_test` 后缀的包）。官方没有强制布局，这四条是社区共识的部分。

7. 用测试驱动开发，Go 的目录结构怎么摆

7.1. 费曼直觉

Go 在测试这件事上是最省心的语言之一：不需要新建 `test/` 目录、不需要引入测试框架、不用改构建配置、没有“测试源码根目录”这种概念。规则只有一条——

> 📌 **核心要点 测试文件放在被测代码旁边，文件名加 `_test.go` 。**目录结构只由“你要分成几个包”决定，测试文件跟着包走，不单独占地方。

7.2. 硬核机制

7.2.1. 最小结构：一个包

```
myapp/
├── go.mod
├── kv.go          ← package myapp
└── kv_test.go     ← package myapp（同目录、同包）
```

Listing 27: 最小 TDD 结构：两个文件、一个目录、零配置

就这么简单。`go test ./...` 会自动找到 `kv_test.go` 并运行，不需要在任何地方“注册”测试。

7.2.2. 多包时的结构：包在哪，测试就在哪

```
db-go/
├── go.mod
├── cmd/
│   └── dbshell/
│       └── main.go            ← package main（入口，一般不需要测试）
└── internal/
    └── db/
        ├── kv.go              ← package db
        ├── kv_test.go         ← package db（与实现同包）
        ├── table.go
        └── table_test.go
```

Listing 28: 多包项目：每个包目录自带一个 `_test.go` ，测试不集中放置

注意规律：**没有“测试目录”这一层。**标准库自己也是这么组织的——`net/http` 目录里实现文件与测试文件混在一起，一视同仁。

7.2.3. TDD 循环在 Go 里长什么样（实测）

我搭了个最小工程跑完整循环，真实输出如下：

```
# ① 先写测试：引用还不存在的 KV
$ go test ./internal/db/
internal/db/kv_test.go:10:8: undefined: KV
FAIL  example.com/tdd1/internal/db [build failed]

# ② 补上实现
$ go test ./internal/db/
ok    example.com/tdd1/internal/db  0.003s

# ③ 什么都不改，再跑一次
$ go test ./internal/db/
ok    example.com/tdd1/internal/db  (cached)

# ④ 看每个测试的名字
$ go test -v -run TestKVBasic ./internal/db/
=== RUN   TestKVBasic
--- PASS: TestKVBasic (0.00s)
PASS
```

Listing 29: TDD 的四步：红灯 → 绿灯 → 命中缓存 → 看细节

这里有个心态转换值得点出来：`undefined: KV` **在 TDD 里不是"错误"，而是起点**。Go 编译器会精确告诉你还缺哪个名字、缺哪个方法，把它当 TODO 列表用就行——先把测试写出来（哪怕当场编译不过），再顺着报错一条条补齐。

7.2.4. 两条命名规则

`go help testfunc` 的原文：

```
The 'go test' command expects to find test, benchmark, and example functions
in the "*_test.go" files corresponding to the package under test.

A test function is one named TestXxx (where Xxx does not start with a
lower case letter) and should have the signature,

  func TestXxx(t *testing.T) { ... }
```

两条缺一不可：**文件**必须以 `_test.go` 结尾，**函数**必须叫 `TestXxx`（`Xxx` 不能以小写字母开头）。同一段帮助还列了另外三种：`BenchmarkXxx(b *testing.B)`、`FuzzXxx(f *testing.F)`、`ExampleXxx()`。

至于包名，同目录的测试文件声明 `package db` 就行——和实现同属一个包，代码里直接用 `KV{}`，**既不用 import 也不用加前缀**。（想强制自己只测导出 API 时才需要另一个选项 `package db_test`，那是后面的话题。）

7.2.5. 常用命令速查

命令	作用
`go test ./...`	跑所有包的测试
`go test ./internal/db/`	只跑一个包
`go test -run TestKVBasic ./internal/db/`	只跑名字匹配的测试，支持正则
`go test -v`	打印每个测试的 RUN / PASS 行

`go test -count=1`	禁用结果缓存，强制重跑
`go test -cover`	显示语句覆盖率
`go test -failfast`	第一个失败的测试就停下

⚠️ **陷阱预警 测试函数名以小写开头会被静默忽略。**实测：把 `TestXxx` 写成 `testXxx` 后再跑 `go test -v`，输出里根本没有它，只报 `[no tests to run]`——不报错、不警告、不影响退出码。当你写下测试却"总是通过"时，第一个要检查的就是函数名首字母。

7.3. 陷阱与易错

1. **在 `test/` 或 `tests/` 目录里写单元测试。**Go 没有这个惯例，硬要这样做，那些文件会自成一个叫 `test` 的包，反而必须 import 被测包、处处加前缀——纯粹是自己给自己加活；
2. **以为改了代码还会命中缓存。**`(cached)` 只在包内容与依赖都没变时出现，想确认行为就加 `-count=1`；
3. **同一个目录放两个普通包。**一个目录只能有一个普通包名，唯一例外是 `foo` 与 `foo_test` 这一对；
4. **把慢测试和快测试混在一起。**Go 用 `t.Skip` 或构建标签（build tag）区分，而不是靠目录分层。

7.4. 关键点

📌 **核心要点** Go 的 TDD 结构一句话：**测试跟着包走。**一个包一个目录，测试文件躺在实现文件旁边，文件名 `_test.go`、函数名 `TestXxx`、包名与实现相同。整套约定零配置，`go test ./...` 就是全部。

7.5. 变式检验

1. **换成表驱动测试**：把多个用例装进一个切片再循环 `t.Run`，这是 Go 社区的标准写法（官方 wiki 有专页 TableDrivenTests），写 `Get` / `Set` / `Del` 的边界用例时会大量用到，见下一小节的示例；
2. **换成需要外部数据的测试**：夹具放进同目录的 `testdata/` 子目录——这个目录名是工具链唯一"看不见"的（`go help test` 有原文），放什么都不参与编译；
3. **换成跨包联动的测试**：如果测的是"整个二进制跑起来对不对"，那才轮到 `test/` 目录登场（e2e 测试）；单包单元测试不要用它。

7.6. 同类归类

1. 与 Java 的对照：JUnit 默认把测试放在 `src/test/java` 的镜像目录树里，配合 Maven/Gradle 的约定；Go 反其道而行——**不镜像、不分离**，就在旁边；
2. 与 Python 的对照：pytest 递归收集 `test_*.py`，放哪儿都行（于是需要额外约定）；Go 用位置（同目录）加命名（`_test.go`）双重限定，简单且不会误收；
3. 往下延伸：本课程后面的 KV、SSTable、归并迭代器都适合先写测试再实现——尤其"合并多个有序来源"这类逻辑，表驱动测试能把各种边界一次性钉死。

7.7. Go 语言机制补充

7.7.1. 为什么 Go 不需要单独的测试目录

`go help test` 里有一句：`'Go test' recompiles each package along with any files with names matching the file pattern "*_test.go"`。

测试文件是**按包**被发现的：go 命令遍历包目录，把同目录下 `_test.go` 结尾的文件一并编译进测试二进制。这套机制天然要求“测试与包同目录”，所以 Go 从未有过、也不需要 `test/` 这种约定。

7.7.2. 表驱动测试：社区的标志性写法

把用例写成数据，循环用 `t.Run` 逐个跑，失败时输出用例名——这是 Go 测试的“普通话”：

```
func TestKVGet(t *testing.T) {
  tests := []struct {
    name   string
    preset bool   // 是否先写入这个键
    wantOK bool
  }{
    {"键存在", true, true},
    {"键不存在", false, false},
  }

  for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
      kv := &KV{}
      if err := kv.Open(); err ≠ nil {
        t.Fatal(err)
      }
      if tt.preset {
        kv.Set([]byte("k1"), []byte("v1"))
      }
      _, ok, err := kv.Get([]byte("k1"))
      if ok ≠ tt.wantOK || err ≠ nil {
        t.Fatalf("ok=%v err=%v, 期望 ok=%v", ok, err, tt.wantOK)
      }
    })
  }
}
```

Listing 30: 表驱动测试：用例是数据，t.Run 给每个用例一个名字

好处很直接：加一个边界用例只需在切片里加一行，而 `go test -run 'TestKVGet/键不存在'` 还能单独跑某一个用例。

7.7.3. 没有内置的 watch 模式

`go test` 没有 `--watch` 之类的选项。TDD 想要“改一下、跑一遍”的自动循环，得靠外部工具（gotestsum、entr）或编辑器的测试集成——这属于工具选择，不属于语言约定，所以不必纠结。

> 📌 **核心要点** 一句话收束：**Go 的测试位置由包决定，不由目录约定决定。** 包在哪，`_test.go` 就在哪；`go test ./...` 自己会找上门，你要做的只是把函数名写成 `TestXxx`。

8. 官方仓库里的 test/ 目录是怎么回事

8.1. 费曼直觉

这个观察没错：`golang/go` 仓库根目录确实有一个 `test/` 目录（本机 GOROOT 里就有 `/usr/lib/go/test/`，392 个文件）。但它装的**不是标准库的单元测试**，而是 **Go 工具链自己的测试用例集**——测的是编译器、运行时、链接器这些“造工具的工具”。

那个目录的 README 只有 19 行，其中一句正好回答本问题：

```
The test directory contains tests of the Go tool chain and runtime.
It includes black box tests, regression tests, and error output tests.
They are run as part of all.bash.

Standard library tests should be written as regular Go tests in the appropriate package.
```

最后那句是官方的原话：**标准库的测试应当写成常规 Go 测试、放在对应的包里**——也就是上一节讲的“同目录 `_test.go` ”。根目录的 `test/` 另有用途，两者不是一回事。

8.2. 硬核机制

8.2.1. 两份“测试”的形态对照

对比项	仓库根目录的 `test/`	各包目录里的 `_test.go`
测什么	Go 工具链与运行时（编译器、gc、链接器、语法边界）	那个包自己的行为
文件命名	普通 `.go` （实测 356 个），没有 `_test.go`	必须以 `_test.go` 结尾
`_test.go` 数量	**0 个**	例如 `net/http` 有 38 个
函数形态	没有 `TestXxx` ，靠首行指令注释驱动	`func TestXxx(t *testing.T)`
怎么运行	`go test cmd/internal/testdir` （或 `all.bash` ）	`go test ./...`
谁维护	Go 团队，为工具链服务	包作者，为这个包服务

8.2.2. 实测：那 392 个文件到底长什么样

```
$ ls /usr/lib/go/test/ | wc -l
392
$ ls /usr/lib/go/test/*_test.go 2>/dev/null | wc -l
0
$ head -3 /usr/lib/go/test/alg.go
// build

// Copyright 2016 The Go Authors. All rights reserved.
```

Listing 31: 第一个反直觉的事实：这个“测试目录”里一个 `_test.go` 都没有

它们不是给 `testing` 包跑的测试，而是**给编译器看的用例**：文件首行写一条指令，说明“这个文件应该被怎么处理”。

首行指令	实测数量	含义
`// run`	170	编译、运行、期望成功退出（行为测试）
`// errorcheck`	159	编译**应当失败**，且错误信息要匹配文件里标注的期望
`// compile`	21	只编译，不运行
`// build`	7	只检查能否构建
`// dir`	1	涉及多文件/多目录的测试

8.2.3. 为什么它们必须独立存放

看一眼 `// errorcheck` 的实例就明白了——`/usr/lib/go/test/alias2.go` 的开头：

```
// errorcheck

// Test basic restrictions on type aliases.

package p

import (
  "reflect"
  . "reflect"
)
```

Listing 32: 一个故意重复导入的 errorcheck 文件：它编译本该失败，失败才是“通过”

这份代码**故意写错**，它“通过测试”的标志恰恰是编译失败。如果把它放进某个包的目录里，`go build ./...` 一跑就炸——因为它是普通的 `.go` 文件，会参与常规构建。所以这类文件必须待在一个**不参与常规构建、由专门 runner 读取**的地方，那就是 `test/`。

驱动它们的 runner 也确实存在，就是 README 里点名的那个包（本机 `/usr/lib/go/src/cmd/internal/testdir/testdir_test.go`）。换句话说：`test/` 里的文件只有**这一个**特殊的测试程序认识，`go test ./...` 对它们一无所知。

8.2.4. 对照：标准库包的测试在哪

本机数一下就知道，它们老老实实待在包旁边：

```
$ ls /usr/lib/go/src/net/http/*_test.go | wc -l
38
$ grep -l '^package http$'      /usr/lib/go/src/net/http/*_test.go | wc -l
19
$ grep -l '^package http_test$' /usr/lib/go/src/net/http/*_test.go | wc -l
19
```

Listing 33: net/http 的 38 个测试文件全在包目录里，19 个内部测试 + 19 个外部测试包

> 📌 **核心要点** Go 仓库里其实有**两套**测试设施，别混为一谈：一套是**每个包自己的常规测试**（同目录 `_test.go`，用 `go test` 跑，标准库和应用项目都用它）；另一套是**工具链的合规性测试台**（根目录 `test/`，靠指令注释驱动，由专门的 runner 执行）。你看到的是后者，它服务的是“Go 编译器本身”，不服务任何具体的库。

8.3. 陷阱与易错

1. **看到官方有 `test/` 就跟着建**。那是工具链项目才需要的设施：`//errorcheck` 这类用例根本无法在常规构建里存活，而普通应用没有这种需求；
2. **以为标准库不写单元测试**。恰恰相反，标准库的测试密度很高，只是它们待在各自包里（`net/http` 一个包就 38 个测试文件）；
3. **把“测试目录”和“测试数据目录”搞混**。`testdata/` 是另一回事：那个名字会让 go 工具链**完全跳过**该目录（`go help test` 有原文），专门用来放夹具（fixture）文件。

8.4. 关键点

> 📌 **核心要点** 判断一个 `test/` 目录该不该建，看它**测什么**：测“某个包的行为”→ 测试文件放进那个包（常规规则）；测“整个工具链/编译器的行为”→ 才需要独立的测试台 + 专门的 runner。你的数据库项目属于前者。

8.5. 变式检验

1. **换成 Go 自己的 `cmd/` 命令**：`go` 命令的行为测试放在 `src/cmd/go/testdata/`（每个包自己的测试数据），与根目录的 `test/` 又是两处不同位置——同样是“测试跟包走”；
2. **换成语言服务器/gopls**：它的语法边界用例也是放在自己的测试目录里，由 `go test` 常规驱动——只有当测试对象是“整个程序”时才需要独立的用例目录；
3. **换成本课程的项目**：等你实现到 SQL 解析器时，可以给“一堆 SQL 输入 → 期望的 AST/错误”写表驱动测试，仍然放在 `internal/db/` 里，不需要建 `test/`。

8.6. 同类归类

1. 与 LLVM 的对照：LLVM 也有一套 `lit` 测试（`llvm/test/`），里面是带 `RUN:` 指令的 `.ll` 文件，由专门的 runner 执行——和 Go 的 `test/` 是同一种设计（用指令注释描述期望行为）；
2. 与 Rust 的对照：Rust 编译器有 `tests/ui/` 目录，靠 `//~ ERROR` 注释标注期望的错误，与 Go 的 `// ERROR` 注释几乎一模一样；
3. 往下延伸：这三者都在解决同一个问题——**“测试对象是编译器本身”**。当你测的东西不是某个库的函数，而是“一段输入应该产生什么诊断”时，普通单元测试框架就不够用了，需要“用例 + 期望输出”这种数据驱动的形态，这也正是表驱动测试在更高层次的翻版。

8.7. Go 语言机制补充

8.7.1. 测试的三个层次

把前面几节串起来，Go 里的“测试”其实分三层，各自有各自的容器：

层次	测什么	放在哪 / 怎么跑
单元测试	单个包的行为	包目录里的 `_test.go`，`go test ./...`

集成 / 端到端测试	多个包、整个二进制的联动	`test/`、`e2e/` 目录（应用项目自定），如 Kubernetes
工具链合规性测试	编译器、运行时、语言边界	Go 仓库根目录 `test/` + 专门的 runner

8.7.2. `testdata/`：被工具链无视的目录

`go help test` 原文：

```
The go tool will ignore a directory named "testdata", making it available to hold ancillary data needed by the tests
```

它不参与编译，放任何文件都行——测试要用的输入文件、期望输出、二进制样本都放这儿。本课程后面写 SSTable 时，测试用的数据文件就该放 `internal/db/testdata/`。

> 📌 **核心要点** 一句话收束：**`test/` 不是 Go 的测试惯例，而是“测试对象是整个工具链”时的特例容器。**常规测试跟包走；只有当被测试的东西没有对应的包可依附（比如编译器行为）时，才需要自建测试台。

9. go vet 通过 ≠ 能编译

9.1. 费曼直觉

这是在把测试文件并入 `internal/db/` 的过程中撞出来的反直觉现象：同一个包，`go vet ./...` 退出码 0、一声不吭；`go build ./...` 却报三处 `missing function body`。

原因一句话：**做检查的人和编译代码的人不是同一拨。** `go vet` 依靠 `go/types` 做类型检查，真正编译走的是 `cmd/compile`；两者对“没有函数体的函数”态度完全不同——前者放着不管，后者当场报错。

9.2. 硬核机制

9.2.1. 实测现象

```
$ go vet ./... ; echo "EXIT=$?"
EXIT=0

$ go build ./...
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:14:6: missing function body
internal/db/kv.go:15:6: missing function body
internal/db/kv.go:16:6: missing function body
```

Listing 34: 同一个包：vet 干净退出，build 报三处错

9.2.2. 为什么会有这个差异

工具	背后是谁	对“无函数体”的态度
`go vet`	`go/types`（标准库的类型检查器）	当作“实现在别处”，直接跳过函数体检查
`go build` / `go test`	`cmd/compile`（编译器，noder 阶段）	逐个核对，不是 linkname/wasmimport 就报错

`go/types` 的源码里那句注释写得很清楚（`/usr/lib/go/src/go/types/decl.go`）：

```
	// function body must be type-checked after global declarations
	// (functions implemented elsewhere have no body)
	if !check.conf.IgnoreFuncBodies && fdecl.Body != nil {
```

Listing 35: go/types：没有函数体 = 在别处实现，于是不检查

真正报错的地方在编译器里（`/usr/lib/go/src/cmd/compile/internal/noder/writer.go:1180`）：

```
    } else {
      if base.Flag.Complete || decl.Name.Value == "init" {
        // Linknamed functions are allowed to have no body. Hopefully
        // the linkname target has a body. See issue 23311.
        // Wasmimport functions are also allowed to have no body.
        if _, ok := w.p.linknames[obj]; !ok && wi == nil {
          w.p.errorf(decl, "missing function body")
        }
      }
    }
```

Listing 36: 编译器 noder 阶段：只有 linkname 与 wasmimport 这两类函数被允许没有函数体

这段代码顺带把规范的“例外”落到了实处——规范说“实现在 Go 之外的函数可以省略函数体”，编译器里对应的就是 `//go:linkname` 与 wasmimport 这两个白名单；此外都不行。

> 📌 **核心要点** 两个工具解决的是不同问题：`go/types` 只做**类型是否自洽**的检查（它连函数体都不进）；编译器还要确认**每个声明都有兑现**。所以 `go vet` 干净不代表能编译，反之 `go build` 干净也不代表 vet 没事——它们各有各的检查项，不能互相替代。

9.2.3. 那 `go vet` 的价值在哪

它专门查编译器不会报、但几乎肯定是 bug 的东西（称为 analyzers）。默认集合里常见的几个：

分析器	查什么
`printf`	格式化字符串与参数类型不匹配（`%d` 后面跟字符串）
`lostcancel`	拿到 `context.CancelFunc` 却没调用，造成泄漏
`copylocks`	复制了含 `sync.Mutex` 的值
`nilfunc`	把 nil 函数当值比较
`tests`	测试函数签名写错、`t.Fatal` 用错等

`go help test` 里还有一句相关的：`As part of building a test binary, go test runs go vet on the package and its test source files`——所以跑 `go test` 时，vet 的一部分会自动执行，这也是它能在测试前提示问题的原因。

9.2.4. 可靠的验证顺序

```
    gofmt -l .          # 1. 格式：有输出就是有文件需要格式化
    go build ./...      # 2. 编译：这是唯一能确认「能编过」的命令
    go vet ./...        # 3. 静态检查：查编译器不管的坑
    go test ./...       # 4. 测试：顺带再跑一遍 vet 的子集
```

Listing 37: 四步各管一段，谁也不能替代谁

9.3. 陷阱与易错

1. **把 `go vet` 当“编译前检查”**。顺序反了：vet 自己也要靠类型检查器加载包，包编译不了时它可能照样退出 0（本例就是）；
2. **以为 `go test` 会帮你编译所有包**。它只编译“被测试触及的包”，`cmd/` 里的 main 包若没人引用就不会被检查——要全覆盖就写 `./...`；
3. **被“exit code 0”迷惑**。`go vet` 的 0 只代表“分析器没发现问题”，不代表编译成功。

9.4. 关键点

> 📌 **核心要点** 判断“能不能编译”只有一条路：`go build ./...`（或 `go test ./...`）。`go vet` 是并列的另一项检查，不是前置。两者都做，才算验完。

9.5. 变式检验

1. **换成 IDE 里的红波浪线**：gopls 也分两层——加载错误（BrokenImport）来自 go/packages，类型错误来自 go/types，而 `missing function body` 这类编译错误它同样看不到，得等真正构建才暴露；
2. **换成 `go run`**：它内部会先编译，所以能抓到这个问题，报错与 `go build` 一致；
3. **换成 `go install`**：同样走编译器，一样会拦下。

9.6. 同类归类

1. 与 C 的对照：`gcc -fsyntax-only` 通过不代表能链接——语法检查与链接是两个阶段；Go 这里 vet/build 的关系类似；
2. 与 TypeScript 的对照：`tsc --noEmit` 只做类型检查，打包器（esbuild）另有自己的检查与限制；“类型检查器与编译器是两套实现”是这类语言共同的现实；
3. 往下延伸：本课程后面自己写 SQL 解析器时也会遇到同类分层——词法/语法分析能通过，不等于语义检查（表是否存在、列类型是否匹配）能通过。把检查分层、各报各的错，是可维护设计的常规做法。

9.7. Go 语言机制补充

9.7.1. 为什么 Go 有两套类型检查器

历史与工程原因都有：`go/types` 是标准库的一部分，供**工具**使用（vet、gopls、静态分析）；编译器的类型检查器（`cmd/compile/internal/types2`）起初与它同源，后因性能与内部需求分化，但两者在语言语义上保持一致。差异只出现在这类“工具不必管、编译器必须管”的检查上——本例的 `missing function body` 就是典型：对分析工具来说，“函数体在哪里实现”无关紧要；对要生成机器码的编译器来说，这是必须落实的事。

> 📌 **核心要点** 一句话收束：**`go vet` 查的是“像不像 bug”，`go build` 查的是“能不能编译”**。两者是并列项，不是先后顺序——本项目里一条 `go build ./...` 就能立刻分清“纯风格问题”与“真的编不过”。

Index of Listings

Listing 1 gopls 报错原文的四层结构 7
Listing 2 复现缺失 require 时的原始文案 7
Listing 3 补登记：go get / go mod tidy 7
Listing 4 让编辑器丢掉旧快照 7
Listing 5 用标准库的判定函数实测导出规则；go/token.IsExported 的实现就是“首字符是否为 Unicode 大写字母” 10
Listing 6 internal 规则的两次实测：跨模块被拦，同模块放行 11
Listing 7 本仓库 go.mod 的全部内容 14
Listing 8 go mod edit -json：把 go.mod 摊成结构化字段，Indirect: true 就是那行注释的真实身份 15
Listing 9 go 行是硬性下限：工具链版本不够，直接拒绝构建 15
Listing 10 间接依赖的来龙去脉，从你的测试包一路读到 YAML 库 16
Listing 11 go mod graph 的真实输出：这是“谁声明了谁”，不是构建依赖清单 17
Listing 12 图里有、实际不需要——这是两回事 17
Listing 13 go.sum 记录的才是“会被读到的模块” 17
Listing 14 四次 go vet 的真实输出：每修一层，报错就往里推一层 21
Listing 15 三个条件在代码里的对应位置 22
Listing 16 遮蔽：内层块的同名声明会挡住外层，这也是“名字解析从内往外”的直接后果 24
Listing 17 import 的两种特殊写法：别名与副作用导入 24
Listing 18 包从 import 进来，类型名出现在使用处——两个位置分工明确 26
Listing 19 import 的四种写法；第三种合法但不推荐，第四种常见于驱动注册 28
Listing 20 missing function body：编译器在解析阶段就拒绝，连类型检查都没走到 29
Listing 21 同一个命令，加了 import 之后报的东西完全不同——依赖把错误带进了视野 30
Listing 22 补上函数体后的 Get：从 map 取值，找不到时 ok 为 false 而不是报错 30
Listing 23 标准库顶层：cmd/ 放命令，其余目录各自是一个包 34
Listing 24 net/http 目录里：19 个内部测试文件 + 19 个外部测试包文件，同在一个目录 34
Listing 25 外部测试包与实现同目录，go test 正常工作 35
Listing 26 cannot refer to unexported field：外部测试包只能看导出 API 35
Listing 27 最小 TDD 结构：两个文件、一个目录、零配置 38
Listing 28 多包项目：每个包目录自带一个 `_test.go`，测试不集中放置 38
Listing 29 TDD 的四步：红灯 → 绿灯 → 命中缓存 → 看细节 39
Listing 30 表驱动测试：用例是数据，t.Run 给每个用例一个名字 41
Listing 31 第一个反直觉的事实：这个“测试目录”里一个 `_test.go` 都没有 43
Listing 32 一个故意重复导入的 errorcheck 文件：它编译本该失败，失败才是“通过” 44
Listing 33 net/http 的 38 个测试文件全在包目录里，19 个内部测试 + 19 个外部测试包 44
Listing 34 同一个包：vet 干净退出，build 报三处错 47
Listing 35 go/types：没有函数体 = 在别处实现，于是不检查 47

Listing 36 编译器 noder 阶段：只有 linkname 与 wasmimport 这两类函数被允许没有函数体 48
Listing 37 四步各管一段，谁也不能替代谁 48